OmniLayout: A Schematic-Coupled Multimodal Benchmark for Constraint-Aware Geometric Reasoning in PCB Layout

Taiting Lu[1], Kaiyuan Lin[1], Mingjia Wang[2], Haolin Ye[1], Runze Liu[1], Yuxin Tian[3], Vahe Melkonyan[2], Haoyu Wang[2], Muchuan Wang[1], Chufan Hong [3], Yifan Yang [4], Sung-Liang Chen [2], Yi-Chao Chen [2], Yincheng Jin[5], Mahanth Gowda[1†],
[1]Pennsylvania State University, [2]Shanghai Jiao Tong University, [3]Independent Researcher, [4]Microsoft Research, [5]Binghamton University

Abstract

Recent large multimodal models (LMMs) have demonstrated remarkable progress in 3D spatial reasoning, spatial grounding, and fine-grained geometric understanding. However, their ability to reason about densely packed object placement under strict spatial and functional constraints remains largely unexplored, despite being a fundamental challenge in practical electronic design automation (EDA) workflows. To bridge this gap, we introduce ***OmniLayout***, the first benchmark designed to evaluate LLMs on printed-circuit-board (PCB) layout placement reasoning under real-world geometric, routing, and connectivity constraints. *OmniLayout* contains 1,681 industrial-grade schematic-coupled PCB layouts and includes four tasks: **(1) geometric reasoning** for IC physical placement, with 77.24*K* placement instances constrained within PCB board boundaries; **(2) routability-aware placement reasoning**, generating physically valid component placements; **(3) relational preservation and reference agreement**, measuring whether placements preserve schematic-derived spatial relationships and reporting agreement with an expert reference layout as an auxiliary diagnostic; and **(4) tool-augmented agentic reasoning**in which models invoke external tools to accomplish Tasks (1)–(3). Our results reveal substantial limitations of current LMMs in PCB layout placement, including weak geometric reasoning, poor routability optimization, and only partial preservation of schematic-derived relationships.

1 Introduction

Driven by the explosive growth in semiconductor technology, modern Printed Circuit Boards (PCBs) now contain hundreds to thousands of components that must be precisely positioned and

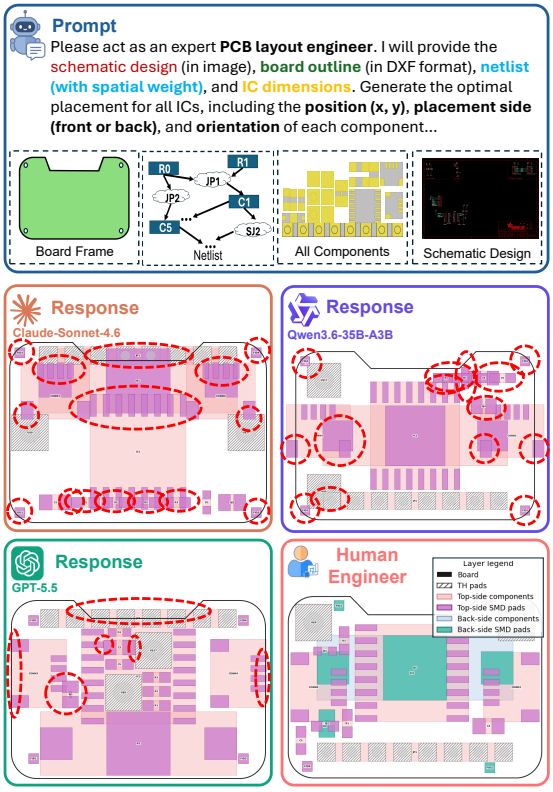


Figure 1: **Large multimodal models (LMMs) fail to perform reliable PCB layout placement under geometric and electrical constraints.** Frequent overlap violations, highlighted by dashed red circles, reveal substantial weaknesses in LMM spatial reasoning and constraint-aware layout generation.

electrically connected within a limited area. Despite this growing complexity, the placement and routing (PnR) process, such as the example shown in Figure 1, remains largely manual, with experienced layout engineers spending several weeks to complete a single design. However, real-world PCB layout designs are almost exclusively proprietary and rarely publicly available. Even within the academic community, most PCB PnR works (Zhang et al., 2025a; Chen et al., 2025b; Han et al., 2026a,b) are trained and evaluated on closed-source private datasets containing only 10 to 20 samples, severely hindering reproducible bench-

† Corresponding author.
Project Page: https://www.omnieda.com/

Findings of the Association for Computational Linguistics: EMNLP 2026, pages 29121–29147
October 24-29, 2026 ©2026 Association for Computational Linguistics

marking and generalization.

Table 1: Comparison of publicly available PCB layout benchmarks and datasets.

Dataset	Layout	Ref. Layout	Sch.	Annotation			Sem. Attr.	DRV	#Layers	Year
				#Comp.	#Pins	#Nets				
Cypress (Zhang et al., 2025a)	10	✗	✗	1.2K	9.5K	2.6K	✗	✗	2	2025
***OmniLayout* (Ours)**	**1681**	✓	✓	**77.24K**	**168.8K**	**139.4K**	✓	✓	2-8	2026

In recent years, large multimodal models (LMMs) (Mouselinos et al., 2024; Chen et al., 2024; Cheng et al., 2024; Rodionov et al., 2025; Huang et al., 2025; Lu et al., 2026a), have demonstrated strong capabilities in 3D spatial reasoning, fine-grained geometry, spatial grounding, and spatial planning. These advances suggest new opportunities for applying LMMs to engineering workflows, particularly for PCB PnR under geometric and electrical constraints. However, these existing spatial-LMM works largely overlook the fundamental challenge of placing densely packed objects within tightly constrained spaces while simultaneously satisfying complex geometric and functional requirements. This raises a critical open question: ***Can current LMMs generate PCB layouts that are geometrically legal, routing-aware, and consistent with schematic-derived circuit relationships across diverse real-world designs?***

To answer this question, we designed a set of experiments to evaluate the PCB PnR capabilities of several state-of-the-art LMMs, including Claude-Sonnet-4.6 (Anthropic, 2026), GPT-5.5 (OpenAI, 2026), and Qwen3.6-35B-A3B (Qwen Team, 2026b), benchmarked against experienced human layout engineers. Specifically, we evaluated their ability to perform component placement under the geometric and electrical constraints characteristic of real-world PCB designs. As illustrated in Figure 1, all evaluated models exhibit more than 5 component overlap errors on PCB layout tasks, while consistently confining all components to a single board side rather than leveraging the spatial capacity of the board. These results reveal that current LMMs fundamentally lack the spatial constraint reasoning and dense placement optimization capabilities required for PCB PnR, as they fail to satisfy component non-overlap constraints, multi-layer spatial utilization, and electrical connectivity requirements essential to practical layout design.

To bridge this gap, we introduce ***OmniLayout***, a large-scale benchmark of real-world schematic-coupled PCB layout designs for evaluating LMMs on multimodal constraint-aware geometric reasoning for PCB placement tasks. As shown in Figure 2, *OmniLayout* contains 1,681 real-world PCB layouts paired with corresponding schematics and expert-designed reference layouts, covering diverse board outlines ranging from industrial PCB stackups from 2 to 8 layers and board areas range from 31.53 mm^2 to 22,458.93 mm^2. The benchmark includes 77.24K IC and component placement instances, 168.82K SMDs and pads, and 55.83K netlist graphs with spatial distance weights and connectivity semantic attributes. We first propose a systematic evaluation protocol for assessing PCB layout designs generated by LMMs with respect to routability, geometric legality, and electrical functionality. Our benchmark evaluates four core abilities, including **(i) geometric reasoning** for physically valid IC and component placement within PCB board boundaries, **(ii) routability-aware placement reasoning** for generating routing-efficient layouts under netlist constraints, **(iii) relational-preservation reasoning** for retaining schematic-derived spatial relationships and functional grouping, while using similarity to an expert layout as an auxiliary reference anchor, and **(iv) tool-augmented agentic reasoning** for iterative PCB placement optimization using external layout-status visualization, overlap localization, geometric-constraint checking and HPWL-based routing feedback, and routability analysis tools to guide constraint-aware layout refinement.

Our main contributions are as follows: **(i)**: We introduce ***OmniLayout***, a comprehensive benchmark for evaluating LMMs on PCB layout placement, containing 1,681 real-world schematic-coupled PCB layouts with expert-designed reference placements and systematic evaluation protocols. **(ii)** We systematically evaluate state-of-the-art LMMs on PCB layout placement under multiple settings, including single-zeroshot placement generation and tool-augmented agentic refinement with iterative visual, geometric, and routability feedback. **(iii)** We provide a detailed analysis of LMM capabilities in PCB layout placement, examining performance across geometric legality, routability-aware spatial reasoning, preservation of schematic-derived relationships and agreement with an expert reference layout, and the effectiveness of tool-augmented agentic refinement with iterative layout feedback.

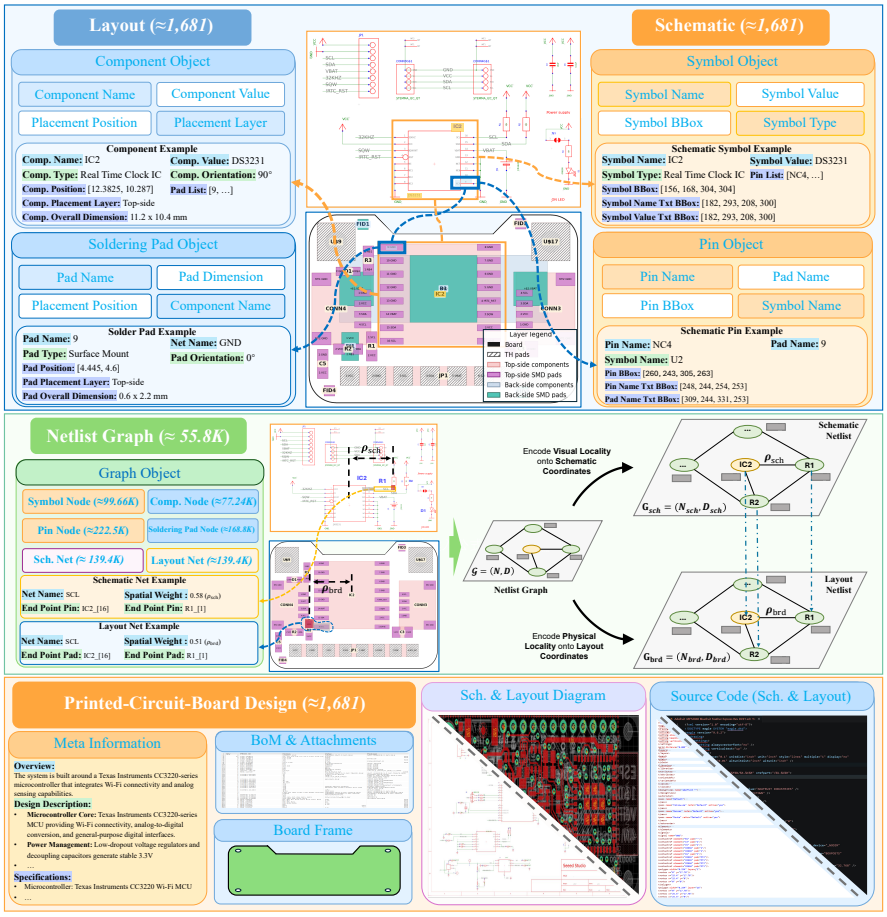
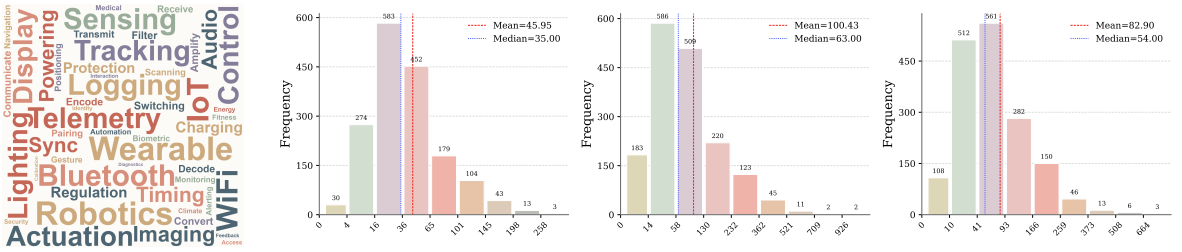


Figure 2: Overview of *OmniLayout* benchmark with representative cases.

2 Related Work

Spatial Reasoning in LMMs. LMMs such as SpatialVLM, SpatialRGPT, FloorplanQA, and Fireplace (Chen et al., 2024; Cheng et al., 2024; Rodionov et al., 2025; Huang et al., 2025; Li et al., 2025) have demonstrated increasingly strong spatial reasoning capabilities, progressing from symbolic and textual spatial reasoning benchmarks such as SPARTQA, SpaRTUN, and StepGame (Mirzaee et al., 2021; Mirzaee and Kordjamshidi, 2022; Shi et al., 2022) toward multimodal and grounded spatial understanding in benchmarks such as SpatialEval, VSI-Bench, Open3DVQA, 3DSRBench, and CA-VQA (Yang et al., 2025; Zhang et al., 2025b; Ma et al., 2025). Recent work further explores spatial world modeling through cognitive-map prompting, depth-aware multimodal learning, and neuro-symbolic reasoning (Yang et al., 2025; Cai et al., 2025; Ali et al., 2025). Despite these advances, existing LMM research has largely overlooked the challenge of placing densely packed objects within tight spatial boundaries while simultaneously satisfying complex geometric and functional constraints.

Benchmarks for PCB Placement. In recent years, a growing number of both commercial tools (Kuo, 2006; Jones, 2023) and academic methods (Zhang et al., 2025a; Chen et al., 2025b; Han et al., 2026a,b) have been proposed to automate the PCB component placement process, spanning classical optimization algorithms, constraint-based solvers, and emerging learning-based approaches. Representative commercial tools such as Cadence Allegro X AI (Kuo, 2006; Jones, 2023), DeepPcb (DeepPCB, 2026) and Quilter (Quilter, 2026) adopt search-based and reinforcement learning strategies respectively to automate PCB component placement. However, these approaches are developed on proprietary private datasets that are never publicly released, and generating a single placement candidate can take up to several hours, posing significant barriers to both reproducibility and practical deployment. Cypress (Zhang et al., 2025a) and ModuPlace (Han et al., 2026b) represent the most recent academic efforts, proposing a VLSI-inspired GPU-accelerated placement framework and an LLM-assisted modular placement method respectively, yet both are evaluated



(a) wordcloud of topics (b) Components per diagram. (c) Pins per diagram. (d) Nets per diagram.

Figure 3: Statistical overview of the *OmniLayout* benchmark. The dataset covers a wide range of electronic domains, with each layout design containing up to 327 components, 1,172 pins, and 840 nets.

on small closed-source or synthesized benchmarks of fewer than 100 components. Current works remain severely limited in both scale and accessibility, as most methods are evaluated on closed-source private datasets containing only tens of designs with limited numbers of components, far below the complexity of real-world industrial PCBs, which fundamentally hinders the generalizability and reproducibility of proposed methods across diverse circuit designs.

3 Benchmark Construction

3.1 Task Formulation

To provide a comprehensive evaluation framework for PCB placement tasks, our task formulation covers four key capabilities of constraint-aware geometric reasoning: **(i) Spatial Reasoning**, which requires determining the position, orientation, and layer assignment of a large number of densely packed components within an irregularly shaped board boundary while satisfying geometric spacing and non-overlap constraints; **(ii) Routability**, which evaluates whether the resulting component placement can be successfully connected by electrical traces without conflicts, a necessary condition for a functional PCB; **(iii) Relational Preservation and Reference Agreement**, which evaluates whether a predicted placement preserves spatial relationships derived from the schematic and reports its similarity to an expert reference layout. Because PCB placement admits multiple valid solutions, reference-layout similarity is treated as an auxiliary diagnostic; **(iv) Agentic Tool Use for Constraint-Aware Geometric Reasoning**, which formulates PCB component placement as an iterative decision-making problem in which LMMs use visualization, overlap checking, boundary checking, and routing feedback to refine their placements.

3.2 Source-driven Annotation Curation

We construct the *OmniLayout* benchmark through a scalable automated pipeline, with full curation details in Appendix A. We gather real-world schematic designs from open-source hardware platforms such as SparkFun, Arduino, Adafruit, and GitHub (SparkFun Electronics, 2026; GitHub, Inc., 2026; Arduino, 2026; Adafruit Industries, 2026; Seeed Technology Co., Ltd., 2026; ProtoCentral, 2026). The collected projects are implemented in Autodesk EAGLE (Autodesk Inc., 2024), a widely used industrial EDA tool that adopts an XML-based file format, allowing efficient parsing of circuit structures and connectivity information through programmatic extraction. To evaluate PCB placement across spatial, routability, and electrical functionality dimensions, we require two types of ground-truth annotations: (i) schematic visual positions encoding intended electrical relationships between components, and (ii) reference layout annotations capturing ground-truth component positions, orientations, layer assignments, and board frame geometry. Rather than costly manual annotation, we leverage two complementary EDA rendering engines. For schematics, we adopt the rendering engine from (Lu et al., 2026b), which parses EAGLE XML files to produce high-fidelity schematic diagrams with pixel-aligned component positions and net-level connectivity annotations, serving as reference signals for evaluating electrical functionality preservation. For reference layouts, while component placements can be extracted directly from EAGLE software, board frame geometry is not captured; we therefore develop a dedicated PCB layout rendering engine that parses source-driven annotations and board frame geometry directly from EAGLE XML layout files containing the reference layout information. Beyond annotation extraction, this engine additionally serves as an interactive visualization tool within our agentic

evaluation framework (Section 5.1), rendering the current placement state as visual feedback to guide LMMs in iteratively refining component positions.

3.3 Statistics of OmniLayout Benchmark

As illustrated in Figure 2 and Figure 3, *OmniLayout* comprises 1,681 real-world schematic-coupled PCB layout designs spanning diverse application domains (e.g., robotics, wireless systems, and sensing), providing a rich and diverse resource for supervised learning in PCB placement and other EDA tasks. Our annotations are organized into three levels: **(i) entity-level annotations** for 77.24K component placements and 168.82K pads, together with their corresponding schematic symbols and pins; **(ii) netlist-graph annotations**, comprising 55.83K graphs that encode circuit topology and connectivity; and **(iii) spatially weighted net annotations**, comprising 139.4K nets represented in both schematic and expert-layout coordinate frames. The 55.83K figure denotes netlist graphs, whereas 139.4K denotes the nets contained within those graphs. Annotation fields and representations are provided in Appendix A.2. These representations support two relational evaluations: schematic relational distance, which measures agreement with spatial relationships derived from the schematic, and reference-layout similarity, which measures agreement with one expert placement.

4 Evaluation Metrics

4.1 Evaluation Protocol

To the best of our knowledge, there is no standardized protocol for evaluating PCB component placement predictions from LMMs that jointly considers geometric legality, routability, and preservation of schematic-derived spatial relationships. PCB PnR task is a well-known non-convex NP-hard problem: given n components to be placed within a constrained board area, the solution space grows combinatorially due to discrete orientations, layer assignments, and non-overlapping constraints, yielding multiple feasible solutions with no efficient algorithm guaranteed to find the global optimum. Consequently, a PCB design generally admits multiple valid placements, and an expert reference layout should not be treated as a unique ground-truth solution. To evaluate a predicted placement against a reference design, we formulate PCB placement evaluation as a constrained optimization assessment problem. Given a set of components V, nets E, and board boundary B, the placement quality is measured by:

$$\min_{\mathbf{x},\mathbf{y},\boldsymbol{\theta},\mathbf{l}} \quad W(\mathbf{x},\mathbf{y},\boldsymbol{\theta}) + \lambda_1 C_{\mathrm{p2p}}(\mathbf{x},\mathbf{y}) + \lambda_2 D_{\mathrm{geo}}(\mathbf{x},\mathbf{y},\boldsymbol{\theta},\mathbf{l}) \tag{1}$$

subject to $\theta_i \in \{0^\circ, 90^\circ, 180^\circ, 270^\circ\}$, $l_i \in \{\text{top}, \text{back}\}$, and $(x_i, y_i) \in B, \forall v_i \in V$, where $W(\mathbf{x},\mathbf{y},\boldsymbol{\theta})$ measures total wirelength as a proxy for routing efficiency, $C_{\mathrm{p2p}}(\mathbf{x},\mathbf{y})$ penalizes pad-to-pad clearance violations, and $D_{\mathrm{geo}}(\mathbf{x},\mathbf{y},\boldsymbol{\theta},\mathbf{l})$ penalizes component overlaps between components assigned to the same board side. The coefficients λ_1 and λ_2 control the relative weights of the clearance and overlap penalties with respect to the wirelength objective. Because PCB placement admits multiple valid solutions, reference-layout agreement is not included in the primary placement objective. Instead, it is reported separately as an auxiliary diagnostic:

$$D_{\mathrm{ref}}(\mathbf{x},\mathbf{y},\mathbf{x}^*,\mathbf{y}^*) = \frac{1}{|E|} \sum_{(v_i,v_j)\in E} \left\| \Delta_{ij}^{\mathrm{pred}} - \Delta_{ij}^{\mathrm{ref}} \right\|_2 \tag{2}$$

where $\Delta_{ij}^{\mathrm{pred}} = (x_i - x_j, y_i - y_j)$ and $\Delta_{ij}^{\mathrm{ref}} = (x_i^* - x_j^*, y_i^* - y_j^*)$ denote the relative spatial displacements of connected component pair (v_i, v_j) in the predicted placement and the expert reference layout, respectively. Lower D_{ref} indicates closer agreement with the relative spatial patterns found in the expert reference. Because PCB placement admits multiple valid solutions, this metric is reported only as an auxiliary diagnostic and should not be interpreted as a measure of electrical correctness or overall placement quality. For readability, the result tables report the corresponding bounded reference-layout similarity:

$$S_{\mathrm{ref}} = \frac{1}{1 + D_{\mathrm{ref}}}, \tag{3}$$

where higher values indicate greater agreement with the expert reference and the reference layout itself has $S_{\mathrm{ref}} = 1$.

4.2 Evaluation Criteria

We define evaluation metrics across four complementary dimensions to comprehensively assess PCB placement quality; full details of each metric are provided in Appendix B due to space constraints. **(i) Geometric Legality**, which evaluates whether the predicted placement satisfies physical constraints, including normalized component overlap area, out-of-boundary ratio, and runtime, collectively measuring whether the placement is

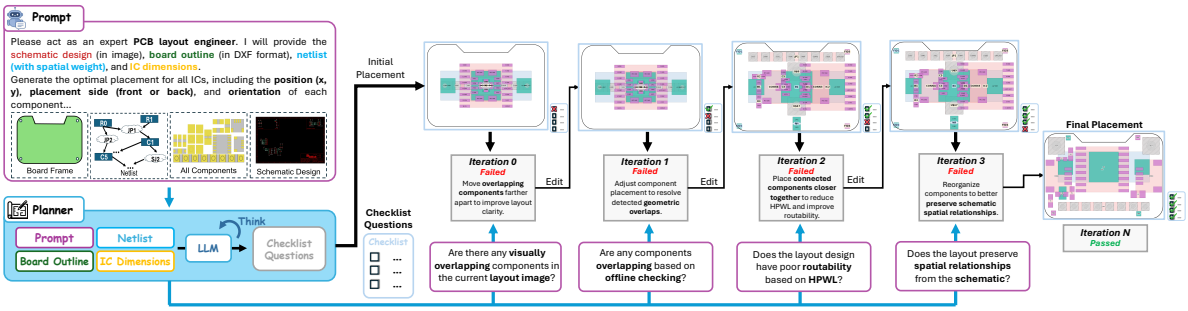


Figure 4: Overview of a multi-modal, multi-agent, multi-round agentic framework for evaluating LMM-based tool usage in PCB component placement.

physically valid within board boundary B. **(ii) Routability**, which assesses routing quality of the predicted placement across both pre-routing proxies — half-perimeter wirelength (HPWL) (Naylor et al., 2001), net crossing (NC) (Zhang et al., 2025a), and net separation (Cheng et al., 2022) — and post-routing metrics obtained via FreeRouting (Freerouting, 2024), including actual routed wirelength (PRWL), routability ratio (Zhang et al., 2025a) and number of vias (#Vias). **(iii) Electrical Functionality Preservation**, which measures whether the predicted placement preserves the relative spatial relationships of subcircuits defined in the circuit schematic. Since PCB placement does not have a unique ground truth solution, we evaluate the degree to which the predicted layout aligns with the subcircuit topology of the schematic diagram in terms of visual relative distances, as well as the component-level relative placement distances of the expert-designed reference layout. **(iv) Agentic Placement**, which evaluates LMMs iteratively refining PCB placement through tool use, measured by: (a) *Constraint Satisfaction Rate*, the ratio of geometric and electrical constraints satisfied at the final placement iteration; (b) *Step Efficiency*, the average number of tool-use steps required to reach a valid placement; and (c) *Placement Refinement Quality*, whether successive placement updates progressively reduce overlap violations, out-of-boundary components, and routing conflicts across iterations.

5 Experiment and Findings

5.1 Experimental Setup

Study Setup. The tested LMMs in this section include GPT-5.5 (OpenAI, 2026), GPT-5-mini (OpenAI, 2025), Gemini-3.1-Pro-Preview (Google DeepMind, 2025b), Gemini-2.5-Flash-Lite (Google DeepMind, 2025a), Claude-Sonnet-4.6 (Anthropic, 2026), LLaMA-4-Maverick (Meta AI, 2025), Mistral-3-14B (Mistral AI, 2025), Qwen3.5-9B (Qwen Team, 2026a), and Qwen3.6-35B-A3B (Qwen Team, 2026b). We evaluate these models alongside one commercial and five open-source PCB placement baselines. We conduct two studies: **(i) Zero-shot Evaluation**, where all LMMs are prompted once without intermediate interaction to assess end-to-end PCB component placement. All settings include the board boundary, netlist, and component dimensions, while we further study the impact of additional information, including schematic images, schematic-aware netlist graphs, global schematic positions, and few-shot examples; and **(ii) Agentic Evaluation**, where LMMs are equipped with layout visualization, overlap checking, and routability verification tools to iteratively refine component placement under geometric and electrical constraints. Implementation details are provided in Appendix C.

Implementation of Commercial and Open-Source PCB Layout Baselines. To comprehensively evaluate the difficulty and quality of our benchmark, we assess placement performance across both commercial and academic baselines. For commercial tool, we evaluate Quilter (Quilter, 2026), a reinforcement learning-based PCB PnR solution. For academic baselines, we implement SA-PCB (OpenROAD, 2024), an OpenROAD-based simulated annealing placer; NS-Place (Cheng et al., 2022), a net separation-oriented analytical placement method; Cypress (Zhang et al., 2025a), a VLSI-inspired GPU-accelerated placement framework; Chen et al. (Chen et al., 2025b), a clearance-constrained global placement method supporting heterogeneous components; and PCBAgent (Chen et al., 2025a), an LLM and reinforcement learning-based placement framework.

Implementation of LMMs Agentic Framework. As shown in Figure 4, we design a multi-

Table 2: **Oneshot evaluation of LMMs on PCB component placement.** *OO* denotes total overlapping area between components; *OoB* denotes components placed outside the board boundary; *RT* denotes inference time in seconds; *HPWL* denotes half-perimeter wirelength; *NC* denotes number of crossings; *NS* denotes average net separation distance; *PRWL* denotes predicted routed wirelength; *RR* denotes fraction of successfully routed nets; *#Vias* denotes number of vias; *Sch. SW-Net* denotes spatial-weight netlist extracted from schematic; *Sch. Img* denotes schematic image; *Sch. Sem.* denotes schematic semantic attributes for components; *Sch.* denotes schematic relational distance; *Ref.* denotes reference-layout similarity.

Model	Size	Input Data	Geometric Legality				Routability							Relational Agreement		Overall
			OO↓	OoB↓	RT↓	P-Pass(%)↑	HPWL↓	NC↓	NS↓	PRWL↓	RR↑	#Vias	R-Pass(%)↑	Sch.	Ref.	E2E Pass↑
Commercial Chatbot Systems																
GPT-5.5	-	Zeroshot	**0.07**	0.17	130.07	72.04	628.78	177.61	**29514.95**	479.03	**0.90**	12.47	15.52	1.189	0.521	15.23
		Sch. SW-Net.	**0.08**	**0.17**	150.19	64.84	623.91	181.50	**28731.48**	390.10	**0.92**	8.98	52.75	0.937	0.593	34.27
		Sch. Sem.	**0.08**	**0.17**	150.56	64.07	**619.65**	178.54	**28141.41**	432.88	0.83	7.05	**79.56**	63.71	0.578	49.08
		Sch. Img	**0.07**	**0.17**	144.93	72.04	628.78	177.61	**29514.95**	435.99	0.91	10.91	**59.55**	1.120	0.559	42.83
		Fewshot	**0.07**	**0.19**	122.46	99.01	**745.75**	**298.28**	**39938.09**	282.75	0.95	1.00	**63.24**	1.063	0.535	**62.61**
GPT-5-mini	-	Zeroshot	0.48	0.24	87.77	98.99	912.87	307.64	127023.27	271.62	0.84	6.40	41.60	1.288	0.486	41.11
		Sch. SW-Net.	0.54	0.24	70.53	98.93	870.62	285.57	105211.90	195.09	0.85	4.40	44.58	0.924	0.531	44.08
		Sch. Sem.	0.54	0.24	71.85	99.29	871.02	289.87	106020.51	253.41	0.81	3.92	60.00	97.03	0.514	56.22
		Sch. Img	0.48	0.24	72.48	98.99	912.87	307.64	127023.27	213.99	0.79	4.87	39.56	1.148	0.494	38.91
		Fewshot	0.38	0.26	66.33	**100.00**	915.73	303.91	79087.22	115.58	0.77	0.20	51.53	1.530	0.437	51.53
Claude Sonnet 4.6	-	Zeroshot	0.49	0.23	10.67	90.18	700.57	177.63	42531.39	245.46	0.76	5.65	43.21	1.161	0.486	27.84
		Sch. SW-Net.	0.49	0.23	10.35	90.30	696.30	177.49	44589.04	209.44	0.86	4.32	50.30	0.923	0.578	45.33
		Sch. Sem.	0.49	0.23	10.38	90.30	702.26	177.88	45734.27	284.77	0.83	4.11	63.19	88.34	0.548	55.56
		Sch. Img	0.49	0.23	10.08	90.18	700.57	177.63	42531.39	253.40	0.85	5.04	49.90	1.041	0.539	44.97
		Fewshot	0.42	0.25	11.47	**100.00**	939.47	313.46	69042.98	190.43	0.54	2.43	53.10	1.315	0.487	53.10
Gemini 3.1 Pro-Preview	-	Zeroshot	0.26	**0.10**	116.10	42.89	672.20	199.23	32692.88	419.16	0.34	1.31	38.59	1.260	0.502	16.55
		Sch. SW-Net.	0.13	0.18	127.28	**99.64**	772.24	281.62	61503.09	387.57	0.91	9.02	**58.32**	1.275	0.552	**58.18**
		Sch. Sem.	0.16	0.19	129.34	99.58	747.65	268.01	52865.44	353.77	0.63	4.75	62.64	1.102	0.552	**62.38**
		Sch. Img	0.24	0.20	118.15	**99.52**	658.02	207.08	32931.45	316.27	**0.92**	6.68	54.94	1.031	0.577	**54.61**
		Fewshot	0.16	0.22	127.48	**100.00**	836.68	332.20	56154.95	97.65	0.73	0.80	56.25	1.199	0.550	56.25
Gemini 2.5 Flash-lite	-	Zeroshot	0.61	0.32	**6.50**	89.53	1080.87	285.58	523166.68	192.76	0.20	5.12	42.89	1.165	0.486	38.40
		Sch. SW-Net.	0.57	0.33	**6.57**	86.67	1085.53	326.74	313629.50	**138.05**	0.79	3.05	41.37	1.196	0.482	35.51
		Sch. Sem.	0.58	0.33	**6.21**	86.44	1118.50	334.70	361803.42	285.80	0.21	4.79	48.16	1.369	0.456	41.63
		Sch. Img	0.61	0.32	**8.43**	89.53	1080.87	285.58	523166.68	164.08	0.83	3.56	40.20	1.026	0.503	35.51
		Fewshot	0.51	0.38	**4.81**	**98.12**	1354.06	332.89	180152.54	131.82	0.62	0.50	37.83	1.530	0.452	37.12
Open-Source MLLMs																
LLaMA 4 Maverick	400B	Zeroshot	0.54	0.28	40.67	**99.46**	1037.73	261.07	368557.83	225.27	0.82	5.15	**43.60**	1.494	0.478	**43.37**
		Sch. SW-Net.	0.53	0.27	32.32	**99.64**	1054.92	274.45	226205.34	195.67	0.83	4.27	38.89	1.296	0.490	38.73
		Sch. Sem.	0.54	0.27	32.91	**99.64**	990.15	269.40	384300.14	251.62	0.83	4.06	58.54	97.38	0.508	55.44
		Sch. Img	0.54	0.28	30.77	99.46	1037.73	261.07	368557.83	226.32	0.83	4.87	40.47	1.244	0.507	40.15
		Fewshot	0.50	0.29	27.94	**100.00**	1080.05	326.79	109768.57	**15.86**	**1.00**	0.50	30.51	1.547	0.449	30.51
Ministral 3 14B	14B	Zeroshot	0.58	0.30	17.33	99.52	1018.66	281.81	269168.06	**182.98**	0.85	4.12	34.77	1.172	0.493	34.62
		Sch. SW-Net.	0.59	0.29	23.79	99.35	960.08	279.51	207435.41	186.27	**0.84**	4.21	35.97	1.115	0.504	35.69
		Sch. Sem.	0.57	0.30	25.42	99.35	946.81	274.31	163720.97	236.36	0.83	3.72	49.55	97.09	0.495	45.69
		Sch. Img	0.58	0.30	19.23	99.52	1018.66	281.81	269168.06	**177.81**	0.85	4.08	41.70	1.234	0.486	41.23
		Fewshot	0.52	0.36	17.24	**100.00**	1047.79	319.97	117151.31	548.91	**1.00**	2.00	52.48	1.350	0.474	52.48
Qwen3.5-9B	9B	Zeroshot	0.59	0.26	460.47	1.61	**460.30**	**90.59**	28168.13	279.67	0.31	6.00	29.17	1.520	0.402	0.47
		Sch. SW-Net.	0.64	0.24	598.24	5.77	891.03	247.08	70491.50	208.04	0.77	2.92	36.36	1.318	0.576	2.14
		Sch. Sem.	0.64	0.22	380.75	6.96	912.23	269.19	92222.56	255.94	0.84	2.92	35.45	6.96	0.521	2.32
		Sch. Img	0.60	0.27	574.55	5.18	**591.87**	**141.90**	51896.65	115.39	0.84	3.22	37.93	1.134	0.492	1.96
		Fewshot	0.43	0.48	389.54	14.52	1646.13	934.79	186513.73	174.26	0.81	0.33	10.46	1.245	0.449	1.52
Qwen3.6-35B-A3B	35B	Zeroshot	0.64	0.13	456.81	13.21	603.41	159.19	42572.41	250.81	0.31	1.50	40.22	1.164	0.516	5.31
		Sch. SW-Net.	0.62	0.21	343.63	51.22	**594.01**	**169.56**	46996.45	147.36	0.81	3.83	45.07	0.642	0.571	22.84
		Sch. Sem.	0.62	0.20	286.55	48.07	634.59	**160.86**	154873.07	**186.55**	0.80	3.00	59.33	46.76	0.522	27.42
		Sch. Img	0.61	0.22	255.28	51.04	620.16	174.54	70062.54	202.15	0.83	3.79	48.24	0.981	0.477	24.45
		Fewshot	0.35	0.21	324.74	92.33	767.92	310.12	56298.48	462.58	0.76	1.50	44.75	1.108	0.537	41.32
Commercial PnR Tools																
Quilter (Quilter, 2026)	-	-	0.023	0.019	5841.00	–	553.48	46.37	1.65	194.28	0.84	163.27	–	1.24	0.57	–
Open-Source PnR Tools																
PCBAgent (Chen et al., 2025a)	-	-	0.41	0.24	316.57	–	1077.03	372.44	109405.37	244.79	0.42	5.51	–	0.30	0.41	–
SA-PCB (OpenROAD, 2024)	-	-	0.22	0.28	40.94	–	1098.81	334.60	129177.56	165.09	0.25	3.44	–	4.22	0.39	–
NS-Place (Cheng et al., 2022)	-	-	0.73	0.23	10.87	–	1010.54	295.80	89299.07	168.45	0.23	3.85	–	1.84	0.38	–
Cypress (Zhang et al., 2025a)	-	-	0.24	0.18	26.83	–	994.54	332.59	128294.67	300.61	0.33	4.90	–	4.68	0.49	–
Chen et al. (Chen et al., 2025b)	-	-	0.19	0.20	131.37	–	602.21	130.03	22950.45	369.99	0.62	6.11	–	3.19	0.86	–
Human Engineer (Ref. Layout)	-	-	0.01	0.14	–	100.00	579.91	114.98	19664.86	140.89	1.00	15.4	–	1.01	1.00	–

modal, multi-agent, and multi-round agentic evaluation framework where LMMs are provided with the board boundary, netlist, component dimensions, and schematic diagram, and equipped with three tools: layout visualization for rendering the current placement state, overlap checking for reporting constraint violations, and routability verification for estimating routing conflicts. At each step, the LMM observes the rendered layout and iteratively refines component positions, orientations, and layer assignments until all constraints are satisfied or the maximum number of steps is reached. Agentic metrics are computed only over successfully parsed placements. API failures and empty or invalid structured responses are reported as unavailable rather than assigned zero-valued metrics.

5.2 Main Results and Findings

Evaluation results of zero-shot study and agentic study are shown in Table 2 and Table 3. Implementation details are provided in Appendix C due to space constraints. Overall, the dense component placement performance of LMMs remains significantly below that of human engineers, highlighting their limited capability in comprehensive layout design. Evaluation results reveal consistent limitations across multiple layout design capabilities.

• **Geometric Legality.** While LMMs demonstrate promising performance on simple PCB layouts with fewer than 40 components per design, their performance degrades significantly as complexity and component density increase. Furthermore, LMMs exhibit poor spatial utilization, placing 80% of components on the top layer rather than distributing them across available routing layers, suggesting a limited understanding of three-dimensional spatial organization in PCB design.

• **Routability.** LMMs exhibit substantial weakness in routability-aware placement, treating component arrangement as a purely spatial packing problem while neglecting circuit connectivity. Although zeroshot LMMs achieve promising score in overlap of less than 10% and out of boundary of less than 10%, their placements consistently result in poor routing outcomes reflected in

Table 3: **Agentic PCB component-placement evaluation.** Lower is better except for RR and Ref. Sim.; NS is reported as a raw value, and Steps is the mean number of refinement steps.

Model	Size	Tool Setting	Geometric Legality			Routability						Relational Placement		Agentic Process
			OO↓	OoB↓	RT↓	HPWL↓	NC↓	NS↓	PRWL↓	RR↑	#Vias↓	Sch. Rel.	Ref. Sim.	Steps
Commercial Chatbot Systems														
GPT-5.5	-	Visualization	**0.015**	0.010	410.276	622.557	167.600	15310.159	113.94	0.591	**0.00**	0.966	0.586	3.900
		Geo. Check	0.003	0.007	295.334	790.055	357.200	35016.412	201.28	0.631	8.00	0.893	0.538	2.200
		Routing Score	0.010	**0.000**	517.918	**281.964**	**82.000**	**2846.929**	163.17	**0.735**	**2.00**	1.000	0.552	4.750
		All Tools	0.005	0.161	496.705	304.824	53.375	3350.575	551.83	0.509	2.00	0.675	0.569	5.875
GPT-5-mini	-	Visualization	0.028	0.045	**120.336**	664.691	179.400	17363.408	387.20	0.370	2.00	1.336	0.499	2.600
		Geo. Check	0.049	0.075	813.394	1039.503	522.200	67860.676	222.05	0.347	7.00	1.569	0.465	16.800
		Routing Score	0.095	0.046	522.033	824.905	392.800	49582.308	530.78	0.200	8.00	1.633	0.429	12.400
		All Tools	0.024	0.293	1362.921	241.864	59.500	2722.143	204.42	0.300	5.00	1.327	0.400	28.750
Claude Sonnet 4.6	-	Visualization	0.029	**0.000**	120.738	288.944	69.714	3775.568	132.57	0.540	3.67	1.344	0.553	4.571
		Geo. Check	0.011	0.028	546.897	790.849	330.400	30869.894	380.68	0.506	7.00	1.029	0.515	11.000
		Routing Score	0.055	0.014	509.824	835.031	421.000	60252.539	270.42	0.614	**2.00**	1.093	0.519	14.000
		All Tools	0.004	0.106	854.905	455.474	97.625	11560.411	343.92	**0.638**	5.50	1.829	0.411	22.625
Gemini 3.1 Pro-Preview	-	Visualization	0.071	0.023	125.666	306.256	69.000	4350.194	**84.35**	0.586	**0.00**	0.920	0.713	1.750
		Geo. Check	0.007	0.030	417.642	1068.203	506.400	77213.345	347.12	0.484	5.00	0.852	0.553	3.400
		Routing Score	0.015	0.033	441.500	1145.583	616.667	65857.878	**112.59**	0.480	4.3	0.942	0.525	3.000
		All Tools	0.009	0.349	536.814	577.776	117.143	13879.121	50.45	0.455	**1.00**	1.070	0.520	5.000
Gemini 2.5 Flash	-	Visualization	0.049	0.441	2138.082	**147.931**	**18.000**	**1666.811**	147.81	**0.667**	4.00	1.372	0.404	30.000
		Geo. Check	0.098	**0.000**	**29.648**	**306.526**	**102.500**	**3383.558**	197.88	**0.677**	7.00	0.957	0.505	1.000
		Routing Score	0.082	**0.000**	864.272	264.859	88.500	3662.414	130.44	0.274	5.00	2.084	0.406	20.000
		All Tools	0.000	**0.000**	**7.802**	**121.400**	**28.000**	**965.265**	49.83	0.172	6.87	1.137	0.760	3.000
Open-Source MLLMs														
LLaMA 4 Maverick	400B	Visualization	0.121	0.141	120.453	373.654	117.400	6369.738	142.07	0.248	4.33	1.413	0.527	5.000
		Geo. Check	0.104	0.061	579.333	393.620	108.000	7751.014	**108.29**	0.599	**3.00**	2.308	0.362	19.667
		Routing Score	0.107	0.050	831.474	1000.486	558.200	80826.012	175.97	0.562	4.50	2.282	0.371	19.800
		All Tools	0.051	0.142	854.743	409.496	95.500	8014.048	**43.92**	0.287	1.00	0.986	0.545	50.000
Ministral 14B	14B	Visualization	0.130	0.053	84.293	691.764	151.429	28092.990	126.68	0.252	0.50	1.237	0.470	3.286
		Geo. Check	0.177	0.145	249.803	881.112	442.750	34241.999	541.25	0.167	8.00	2.056	0.418	9.750
		Routing Score	0.124	0.101	**169.451**	370.257	122.750	7451.601	179.62	0.530	2.50	1.576	0.421	8.250
		All Tools	0.065	0.180	1515.420	865.037	214.667	34773.643	276.47	0.449	3.00	2.012	0.377	50.000
Qwen3.5-9B	9B	Visualization	–	–	–	–	–	–	–	–	–	–	–	–
		Geo. Check	–	–	–	–	–	–	–	–	–	–	–	–
		Routing Score	–	–	–	–	–	–	–	–	–	–	–	–
		All Tools	–	–	–	–	–	–	–	–	–	–	–	–
Qwen3.6-35B-A3B	35B	Visualization	–	–	–	–	–	–	–	–	–	–	–	–
		Geo. Check	–	–	–	–	–	–	–	–	–	–	–	–
		Routing Score	–	–	–	–	–	–	–	–	–	–	–	–
		All Tools	–	–	–	–	–	–	–	–	–	–	–	–

Qwen3.5-9B and Qwen3.6-35B-A3B tool-setting agentic results are unavailable. The supplied placement archive contains unlabelled one-pass Qwen outputs, but does not identify them as Visualization, Geo. Check, Routing Score, or All Tools runs. We therefore report these entries as "–" and exclude them from aggregate calculations; "–" does not denote zero performance. Layout metrics, RT, and Steps are averaged over finalized graph JSON outputs. PRWL and #Vias are reported over fully routed instances, while RR assigns zero to failed routing attempts. The Gemini 2.5 Visualization row uses Gemini 2.5 Flash, while its All Tools row uses Gemini 2.5 Flash-lite and has one finalized placement. Its packaged routing summary does not provide replacement PRWL or #Vias values, so the original table values are retained for those two cells.

higher PRWL and lower RR, suggesting a fundamental gap in understanding the relationship between component placement and downstream routing feasibility.

• **Relational Preservation.** LMMs achieve a similarity score below 60% when comparing predicted placements against reference layout designs, indicating that component positions deviate substantially from both the schematic diagram's topology and the expert-designed reference in terms of relative spatial distances. This quantitative gap reflects a deficiency: LMMs lack a structured understanding of circuit hierarchy and functional grouping, and consequently fail to preserve the spatial relationships of subcircuits during placement.

• **Agentic Tool Use.** Agentic placement is practically important because it allows models to inspect and iteratively repair intermediate layouts. Tool effectiveness is nevertheless model- and metric-dependent. Among the evaluated feedback settings, the All Tools condition yields the lowest overlap error for Claude-Sonnet-4.6, Ministral-14B, and GPT-5-mini (OO $=$ 0.004, 0.065, and 0.024, respectively). However, All Tools does not consistently achieve the best performance on other metrics, such as out-of-boundary violations, routability, and relational agreement. These results indicate that combining multiple feedback channels can improve overlap correction, while effective agentic placement still requires model- and objective-aware tool selection. Additional analysis is provided in Appendix C.1.

• **Agentic Spatial & Routing Reasoning.** By incorporating layout visualization, legality checking, and routability feedback into a multi-agent framework, agentic interaction improves placement performance by reducing component overlap and out-of-boundary violations, while layout visualization feedback contributes to improved routing performance compared to direct generation baselines.

5.3 Diagnostic Analyses

We further conduct additional diagnostic analyses covering constraint-violation frequency, performance across board complexity and density, the contributions of different input representations and feedback channels, and recurring PCB design principles missed by current models.

• **Most frequently violated constraints.** Across the successfully parsed LMM outputs included in this diagnostic, 79.82% contain at least one inter-component pad or SMD overlap, making overlap the most frequent evaluated geometric violation. Boundary-containment violations are similarly widespread: 72.89% of placements contain at least one component block,

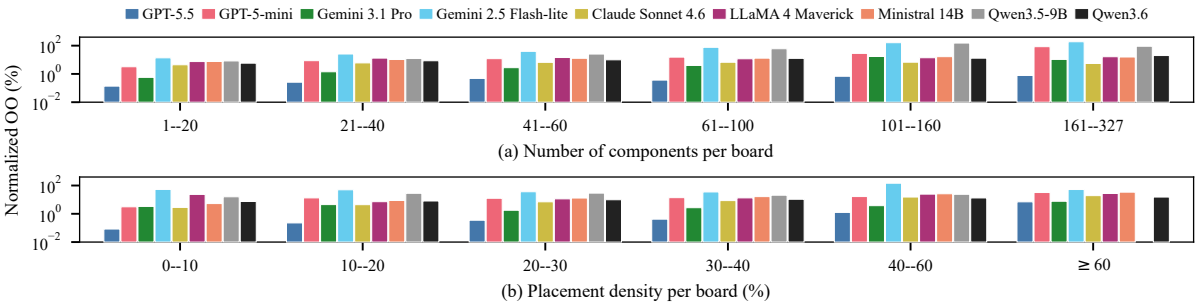


Figure 5: LMM placement performance under increasing PCB complexity: (a) component count per board and (b) placement density. Lower overlap is better.

pad, SMD, or hole extending outside the valid board boundary. These results show that the aggregate OO and OoB scores reflect recurring constraint failures rather than a small number of extreme cases.

• **Impact of board complexity and density.** As shown in Figure 5, we analyze placement performance across boards with component counts from 1 to 327 and placement densities from 0.49% to 90.48%. Complexity is measured by component count, while placement density is the total pad, SMD, and hole area divided by valid board area. As either factor increases, normalized overlap area (OO; lower is better) also increases, with median relative increases of 504% for component count and 337.8% for placement density respectively. Overall, higher board complexity leads to more component overlap and worse placement performance.

• **Contributions of one-pass input representations.** The effectiveness of additional schematic information depends on both the model and the evaluation dimension. For functional grouping, all 15 comparisons across five models and three schematic representations improve over their matched no-schematic conditions. The mean paired AUC gains are +0.079 for the schematic spatially weighted netlist, +0.089 for the schematic image, and +0.091 for semantic component attributes. The best schematic-aware functional-grouping AUC is 0.738 for Gemini 3.1 Pro, 0.727 for Claude Sonnet 4.6, 0.695 for LLaMA 4 Maverick, 0.669 for Ministral 14B, and 0.657 for GPT-5-mini, compared with 0.590–0.646 without schematic context. In contrast, few-shot prompting changes mean AUC by -0.006 and improves only 42.8% of paired-board comparisons. Thus, no input representation is uniformly best across models and metrics.

• **Contributions of agentic feedback channels.** Feedback effectiveness is similarly model- and objective-dependent. As shown in Table 3, Claude Sonnet 4.6 and Ministral 14B achieve their lowest overlap under Geo. Check, where the model receives explicit identities and locations of conflicting components. This structured textual feedback removes the need to infer overlap locations from the rendered image and supports localized corrections. In contrast, GPT-5-mini achieves its lowest overlap under Routing Score, suggesting that scalar HPWL feedback can also guide useful geometric revisions for some models. Combining all feedback channels is therefore not uniformly optimal.

• **Systematically missed PCB design principles.** The diagnostic analyses reveal four recurring weaknesses: **(i)** maintaining pad-level non-overlap, **(ii)** keeping all component geometry within the board boundary, **(iii)** preserving proximity among electrically connected or schematically co-grouped components, and **(iv)** adapting placement strategies to crowded or component-heavy boards. Frontier LMMs occasionally capture these principles, but they do not apply them reliably across boards, input representations, or feedback conditions.

6 Conclusion

We introduce OmniLayout, a benchmark for evaluating LMMs on PCB layout geometric reasoning, consisting of 1,681 real-world PCB layouts coupled with fine-grained annotations and structured evaluation protocols. We evaluate state-of-the-art LMMs under both end-to-end and tool-augmented settings, alongside open-source and commercial PCB placement tools as baselines. Our analysis reveals persistent limitations in geometric legality, routability optimization, agentic refinement, and preservation of schematic-derived relationships.

Limitations

The agentic evaluation does not contain more than 50 turn iterations, primarily due to the substantial computational cost and resource requirements of iterative agentic inference, and thus may miss evaluating placement refinement scenarios that require longer iterative convergence.

We evaluate each of the three tools (layout visualization, overlap checking, and routability verification) in isolation as well as in full combination to assess their individual and collective impact on placement quality. The benchmark does not cover all possible tool combination settings for agentic evaluation; for example, partial subsets such as pairing layout visualization with overlap checking are not explored in this work. It is an extensive research direction and beyond the scope of this work. However, as the benchmark is open-source, we plan to extend it to include broader agentic evaluation settings in the future.

This work focuses on spatial reasoning for PCB component placement, and thus does not study the routing performance of LMMs directly. Instead, we employ FreeRouting as a fixed backbone to evaluate the quality of placements produced by LMMs, isolating placement as the primary variable of interest. Routing optimization remains an important and complementary research direction that we leave for future work.

Ethics Statement

Intended usage. We aim for the benchmark to serve as a comprehensive evaluation platform for assessing both frontier commercial and open-source LMMs and state-of-the-art commercial and academic PnR tools on real-world PCB layout design at scale. The code and dataset are publicly released under the CC-BY-4.0 license, consistent with the licensing terms of all constituent datasets incorporated in the benchmark.

Accessibility and Potential Misuse. The main goal of our work is to encourage the community to use the benchmark to advance PCB layout design research and to highlight the current limitations of existing LMMs and PnR tools on real-world placement challenges. Our evaluations reveal that existing LMMs achieve notably poor performance on PCB layout design tasks, underscoring a significant gap between current model capabilities and the demands of real-world PCB design. However, as the dataset is primarily sourced from DIY and open-source designs, unknown biases inherited from these sources may influence evaluation outcomes and limit the benchmark's coverage of more advanced PCB designs. We tried to mitigate this by incorporating diverse PCB designs from various sources, and we encourage the community to use this benchmark as a foundation to drive future research toward developing models better suited for practical PCB layout design.

Ai Assistants. We used AI-assisted tools to support proofreading and improve the clarity, grammar, and readability of the manuscript. All technical content, analysis, and final revisions were carefully reviewed and validated by the authors.

7 Acknowledgments

This research was supported by the U.S. National Science Foundation (NSF) under CAREER Award No. 2046972. We also gratefully acknowledge support from the Google Gemini Academic Program, which provided access to Gemini models.

References

Adafruit Industries. 2026. Adafruit industries. https://www.adafruit.com.

Momina Liaqat Ali, Muhammad Abid, Muhammad Saqlain, and Jose M Merigo. 2025. Neuro-symbolic control with large language models for language-guided spatial tasks. *arXiv preprint arXiv:2512.17321*.

Anthropic. 2026. Claude sonnet 4.6. https://www.anthropic.com.

Arduino. 2026. Arduino: Open-source electronics platform. https://www.arduino.cc.

Autodesk Inc. 2024. Autodesk eagle: Pcb design and schematic software. https://www.autodesk.com/products/eagle. Accessed: 2026-03-05.

Wenxiao Cai, Iaroslav Ponomarenko, Jianhao Yuan, Xiaoqi Li, Wankou Yang, Hao Dong, and Bo Zhao. 2025. Spatialbot: Precise spatial understanding with vision language models. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9490–9498. IEEE.

Boyuan Chen, Zhuo Xu, Sean Kirmani, Brain Ichter, Dorsa Sadigh, Leonidas Guibas, and Fei Xia. 2024. Spatialvlm: Endowing vision-language models with spatial reasoning capabilities. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14455–14465.

Lin Chen, Ran Chen, Shoubo Hu, Xufeng Yao, Zhentao Tang, Shixiong Kai, Siyuan Xu, Mingxuan Yuan,

Jianye Hao, Bei Yu, and 1 others. 2025a. Pcbagent: An agent-based framework for high-density printed circuit board placement. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference*, pages 781–787.

Yan-Jen Chen, Wei-Kai Huang, Chung-Ting Tsai, Chiao-Yu Ou, and Yao-Wen Chang. 2025b. Clearance-constrained pcb global placement with heterogeneous components. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, pages 1–7. IEEE.

An-Chieh Cheng, Hongxu Yin, Yang Fu, Qiushan Guo, Ruihan Yang, Jan Kautz, Xiaolong Wang, and Sifei Liu. 2024. Spatialrgpt: Grounded spatial reasoning in vision-language models. *Advances in Neural Information Processing Systems*, 37:135062–135093.

Chung-Kuan Cheng, Chia-Tung Ho, and Chester Holtz. 2022. Net separation-oriented printed circuit board placement via margin maximization. In *2022 27th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 288–293. IEEE.

DeepPCB. 2026. Deeppcb: Pure ai-powered, cloud-native pcb routing. https://deeppcb.ai/. Accessed: 2026.

Freerouting. 2024. Freerouting: Advanced pcb auto-router. https://github.com/freerouting/freerouting. Accessed: 2024.

GitHub, Inc. 2026. Github: Software development platform. https://github.com.

Google DeepMind. 2025a. Gemini 2.5 Flash-Lite. https://deepmind.google/technologies/gemini.

Google DeepMind. 2025b. Gemini 3 Pro Preview. https://deepmind.google/technologies/gemini. Accessed: 2025.

Yaohui Han, Beichen Li, Rongliang Fu, Qunsong Ye, Zhiyuan Lu, Junchen Liu, Bei Yu, Tsung-Yi Ho, and Tinghuan Chen. 2026a. Pcb-migrator: Automated pcb pnr migration. In *Proc. DATE*, pages 1–7.

Yaohui Han, Beichen Li, Mingyang Zhao, Rongliang Fu, Qunsong Ye, Tinghuan Chen, Bei Yu, and Tsung-Yi Ho. 2026b. Moduplace: Llm-assisted modular pcb placement via preference-optimized constraint graph generation.

Ian Huang, Yanan Bao, Karen Truong, Howard Zhou, Cordelia Schmid, Leonidas Guibas, and Alireza Fathi. 2025. Fireplace: Geometric refinements of llm common sense reasoning for 3d object placement. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 13466–13476.

Cooper Jones. 2023. *Distributed Monte Carlo Tree Search With Applications To Chip Design*. Ph.D. thesis, Massachusetts Institute of Technology.

Samuel P Kuo. 2006. *PCB Design and Simulation Using Cadence Allegro 15.5*. Ph.D. thesis, University of Illinois at Urbana-Champaign.

Yifan Li, Lichi Li, Anh Dao, Xinyu Zhou, Wenjun Huang, Tianyi Ma, Yicheng Qiao, Zheda Mai, Daeun Lee, Zichen Chen, and 1 others. 2025. Industrynav: Exploring spatial reasoning of embodied agents in dynamic industrial navigation. *arXiv preprint arXiv:2511.17384*.

Taiting Lu, Kaiyuan Lin, Ziwei Dong, Sisong Bei, Haolin Ye, Yuxin Tian, Runze Liu, Mingjia Wang, Jingying Zeng, Hongxing Pan, Kai Zhang, Haoyu Wang, Guoliang Shi, Ling Ma, Yifan Yang, Jiaying Lu, Qi He, Yi-Chao Chen, Sung-Liang Chen, and 2 others. 2026a. Omnirouting: A semantic-coupled multimodal benchmark for constraint-aware spatial reasoning in pcb routing. *Preprint*, arXiv:2608.04434.

Taiting Lu, Kaiyuan Lin, Yuxin Tian, Yubo Wang, Muchuan Wang, Sharique Khatri, Akshit Kartik, Yixi Wang, Amey Santosh Rane, Yida Wang, and 1 others. 2026b. Omnisch: A multimodal pcb schematic benchmark for structured diagram visual reasoning. *arXiv preprint arXiv:2604.00270*.

Wufei Ma, Haoyu Chen, Guofeng Zhang, Yu-Cheng Chou, Jieneng Chen, Celso de Melo, and Alan Yuille. 2025. 3dsrbench: A comprehensive 3d spatial reasoning benchmark. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 6924–6934.

Meta AI. 2025. LLaMA 4 Maverick. https://ai.meta.com/llama.

Roshanak Mirzaee, Hossein Rajaby Faghihi, Qiang Ning, and Parisa Kordjamshidi. 2021. Spartqa: A textual question answering benchmark for spatial reasoning. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4582–4598.

Roshanak Mirzaee and Parisa Kordjamshidi. 2022. Transfer learning with synthetic corpora for spatial role labeling and reasoning. In *Proceedings of the 2022 conference on empirical methods in natural language processing*, pages 6148–6165.

Mistral AI. 2025. Mistral 3 (14B). https://mistral.ai.

Spyridon Mouselinos, Henryk Michalewski, and Mateusz Malinowski. 2024. Beyond lines and circles: Unveiling the geometric reasoning gap in large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 6192–6222.

William C Naylor, Ross Donelly, and Lu Sha. 2001. Non-linear optimization system and method for wire length and delay optimization for an automatic electric circuit placer. US Patent 6,301,693.

OpenAI. 2025. GPT-5-mini. https://openai.com. Accessed: 2025.

OpenAI. 2026. Gpt-5.5. https://www.openai.com.

OpenROAD. 2024. SA-PCB: Simulated annealing for printed circuit board placement. https://github.com/The-OpenROAD-Project/SA-PCB.

ProtoCentral. 2026. Protocentral: Open source medical electronics. https://protocentral.com.

Quilter. 2026. Quilter technology: Automated circuit board design. https://www.quilter.ai/product/technology.

Qwen Team. 2026a. Qwen3.5-9B. https://qwenlm.github.io.

Qwen Team. 2026b. Qwen3.6-35b-a3b. https://huggingface.co/Qwen/Qwen3.6-35B-A3B.

Fedor Rodionov, Abdelrahman Eldesokey, Michael Birsak, John Femiani, Bernard Ghanem, and Peter Wonka. 2025. Floorplanqa: A benchmark for spatial reasoning in llms using structured representations. *arXiv preprint arXiv:2507.07644*.

Seeed Technology Co., Ltd. 2026. Seeed studio. https://www.seeedstudio.com.

Zhengxiang Shi, Qiang Zhang, and Aldo Lipani. 2022. Stepgame: A new benchmark for robust multi-hop spatial reasoning in texts. In *Proceedings of the AAAI conference on artificial intelligence*, volume 36, pages 11321–11329.

SparkFun Electronics. 2026. Sparkfun electronics. https://www.sparkfun.com.

Jihan Yang, Shusheng Yang, Anjali W Gupta, Rilyn Han, Li Fei-Fei, and Saining Xie. 2025. Thinking in space: How multimodal large language models see, remember, and recall spaces. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 10632–10643.

Niansong Zhang, Anthony Agnesina, Noor Shbat, Yuval Leader, Zhiru Zhang, and Haoxing Ren. 2025a. Cypress: Vlsi-inspired pcb placement with gpu acceleration. In *Proceedings of the 2025 International Symposium on Physical Design*, pages 31–41.

Weichen Zhang, Zile Zhou, Xin Zeng, Xuchen Liu, Jianjie Fang, Chen Gao, Yong Li, Jinqiang Cui, Xinlei Chen, and Xiao-Ping Zhang. 2025b. Open3d-vqa: A benchmark for comprehensive spatial reasoning with multimodal large language model in open space. *arXiv preprint arXiv:2503.11094*.

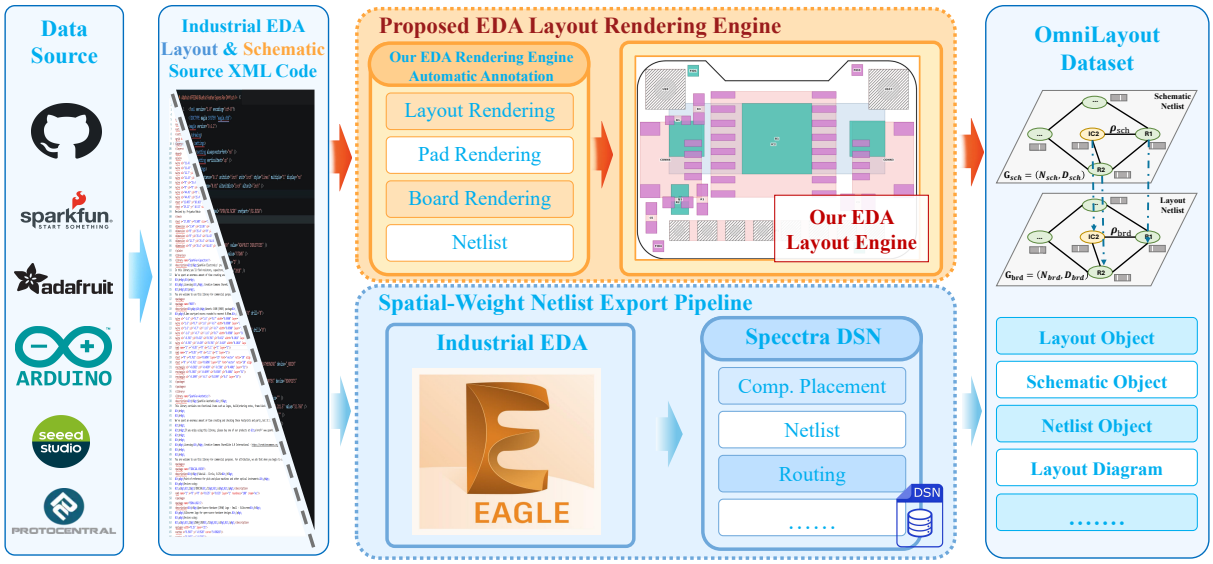


Figure 6: **Automated PCB layout annotation pipeline for *OmniLayout*.** We directly export Specctra DSN files from industrial EDA tools and leverage our EDA layout rendering engine to automatically generate PCB layout annotations, including component placements, orientations, board geometry, routing information, and spatial-aware netlists.

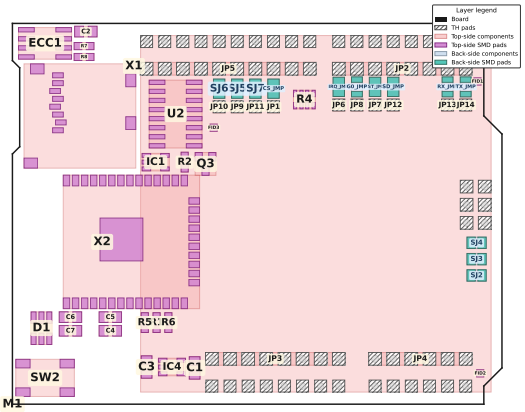
A Data Collection

A.1 Automatic Annotation Pipeline

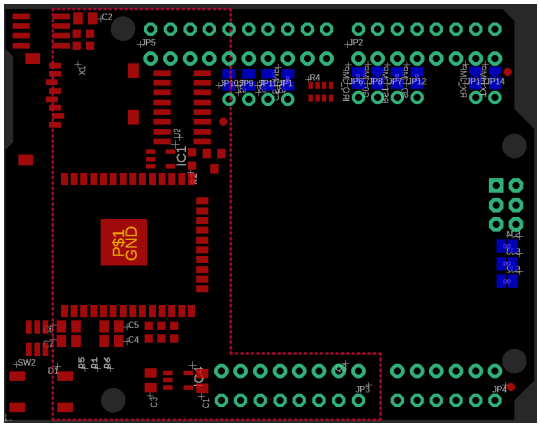
To support scalable benchmark construction, we develop a source-driven annotation pipeline that extracts all required ground-truth information directly from EAGLE XML project files, covering both schematic-side and layout-side annotations without manual image-space labeling. Our pipeline operates on the XML-based format used by Autodesk EAGLE 9.6.2, which explicitly encodes component instances, symbol and footprint definitions, pin attributes, net topology, layer assignments, orientations, and board frame geometry as structured data.

Schematic Annotation. For schematic-side annotations, we adopt the rendering engine from OmniSch (Lu et al., 2026b), which parses EAGLE XML schematic files to produce high-fidelity schematic diagrams paired with pixel-aligned annotations. Given a schematic project, the engine recovers the hierarchical circuit structure including symbol libraries, instance placements, pin attributes, and net topology, and programmatically reconstructs the schematic diagram by reproducing the native EDA visualization of symbols, wires, and text elements. Since each rendered visual primitive remains linked to its underlying semantic entity during image generation, the pipeline automatically extracts image-aligned annotations for symbol instances, pins, textual attributes, net labels, wires, and junctions, along with structural metadata required for downstream topology reasoning. These schematic annotations serve as reference signals for evaluating electrical functionality preservation in placement results.

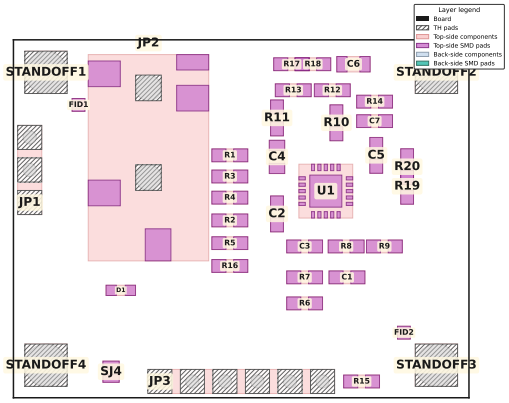
Layout Annotation. For layout-side annotations, we develop a dedicated PCB layout rendering engine that parses ground-truth placement information directly from EAGLE XML layout files. Given a layout project, the engine first recovers the complete placement representation including component instance names, reference footprint definitions, pad-level block dimensions, net topology, and per-component position and rotation attributes. Each component instance is associated with its footprint definition, from which we extract block-level geometry including pad type (`smd`/`pad`/`hole`), physical dimensions, and layer assignment (`front`/`back`/`through`). Board frame geometry is separately recovered from DXF-encoded boundary primitives embedded in the layout file, from which we compute the bounding extents used to constrain all placement coordinates. The engine then programmatically reconstructs the PCB layout diagram, with each rendered element remaining linked to its underlying semantic entity, enabling automatic extraction of pixel-aligned annotations for component bounding boxes, pad locations, layer assignments, rotation codes, net connections, and board boundary primitives.



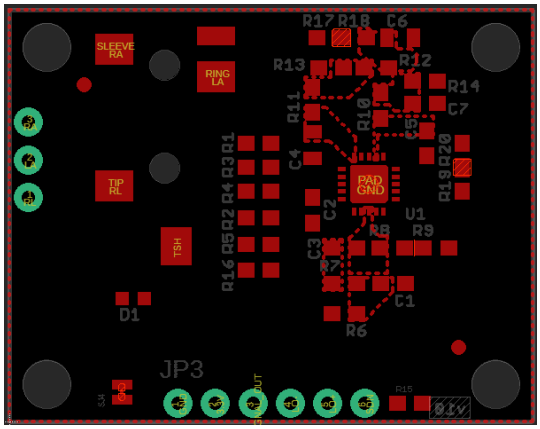
(a) Layout Diagram by Custom EDA Layout Engine



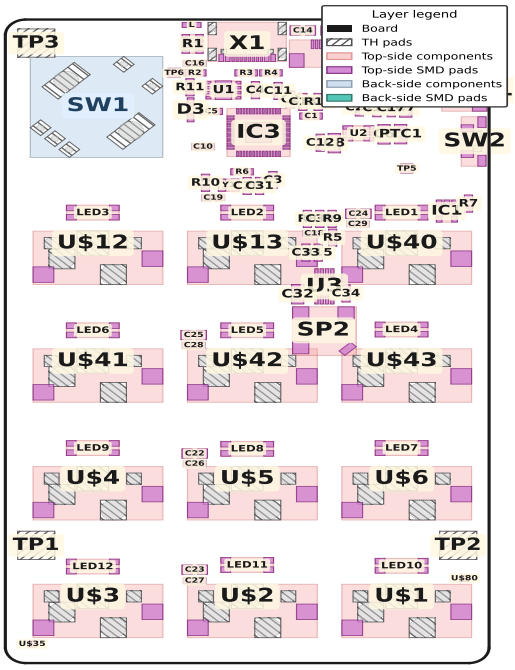
(b) Layout Diagram by Industrial EDA Layout Engine



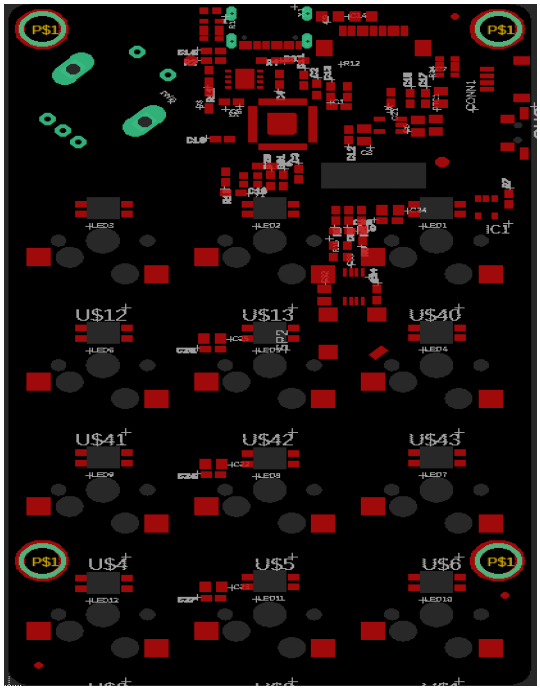
(c) Layout Diagram by Custom EDA Layout Engine



(d) Layout Diagram by Industrial EDA Layout Engine



(e) Layout Diagram by Custom EDA Layout Engine



(f) Layout Diagram by Industrial EDA Layout Engine

Figure 7: Qualitative examples of our custom PCB layout rendering pipeline. Each row corresponds to one layout example. The left column shows layouts rendered by our custom EDA engine, and the right column shows the corresponding reference layouts produced by industrial EDA tools.

Unified Pipeline. Both annotation streams are processed within a unified pipeline that aligns schematic-side connectivity signals with layout-side placement ground truth through shared component instance identifiers. This cross-file correspondence enables automatic construction of paired training and evaluation samples covering spatial, routability, and electrical functionality dimensions without any manual intervention. Compared with purely manual image-space annotation, this source-driven strategy improves annotation consistency, eliminates geometric ambiguity arising from dense component layouts, and scales efficiently to large benchmark collections. Beyond static annotation extraction, the layout rendering engine additionally serves as an interactive visualization tool within our agentic evaluation framework, rendering the current placement state as visual feedback to support iterative placement refinement by large multimodal models.

A.2 Structured Annotation Schema

The annotations in *OmniLayout*are organized into three levels with different annotation objects and counting units.

Entity-level annotations. This level describes individual objects in the PCB layout and schematic. For each layout component, we provide its identity, value, package, reference placement, orientation, layer assignment, and bounding-box information. For each pad, we provide its identity, type, dimensions, position in the component coordinate frame, layer, and associated component. The corresponding schematic symbols and pins are annotated with their identities, positions, textual attributes, and component associations. Shared identifiers establish explicit correspondences between layout components and pads and their schematic symbols and pins. In total, the benchmark contains 77.24K component placements and 168.82K pads.

Netlist-graph annotations. This level represents circuit topology. The benchmark contains 55.83K netlist graphs whose nodes correspond to layout components and pads or their schematic symbols and pins. Graph connectivity records the electrical relationships among these entities, including pin–pad correspondences, pad–pad connections, and semantic net labels when available. These annotations provide an unweighted structural representation of circuit connectivity.

Spatially weighted net annotations. This level augments the graph topology with spatial information. Across the netlist graphs, the benchmark contains 139.4K nets. Each net has two aligned spatial representations: a layout representation computed from component and pad positions in the expert-designed reference layout, and a schematic representation computed from the positions of the corresponding symbols and pins in the schematic coordinate frame. These representations provide layout-based and schematic-based spatial weights for evaluating reference-layout agreement and schematic-relationship preservation, respectively.

The two dataset statistics therefore refer to different objects: 55.83K is the number of netlist graphs, whereas 139.4K is the number of nets contained in those graphs. Spatially weighted annotations measure relational organization and should not be interpreted as verification of electrical correctness.

A.3 Visualization Example of Custom Layout Engine

To qualitatively illustrate the capability of our custom EDA layout rendering engine, we provide representative examples in Figure 7. For each example, we show the corresponding industrial EDA layout rendered by Autodesk EAGLE alongside the synthesized layout rendered by our custom engine. This side-by-side comparison highlights the fidelity of our rendering pipeline in reproducing component placements, pad geometries, board boundaries, and routing traces. In particular, the examples demonstrate that our synthesized layouts faithfully preserve the structural and spatial characteristics of the original industrial designs, while additionally enabling dense instance-level annotation of component bounding boxes, pad locations, layer assignments, net connections, and board boundary primitives.

B Task Formulation Protocol

In this section we introduce the definition of each task, and evaluation metrics for each task.

B.1 Task Definition

We formulate PCB component placement evaluation across the following metrics, each targeting a distinct aspect of layout quality.

Overlap (OO) refers to the normalized intersecting area between component pairs in the predicted placement, measuring the degree to which

components physically conflict with one another on the board.

Out-of-Boundary (OoB) refers to the ratio of component area placed outside the board boundary B, capturing whether the predicted placement respects the spatial extent of the board.

Half-Perimeter Wirelength (HPWL) refers to the sum of bounding box half-perimeters over all nets, serving as a pre-routing proxy for the total wiring cost of the predicted placement. Lower HPWL indicates more compact and routing-friendly component arrangement.

Net Crossing (NC) refers to the total number of intersections between net connections in the predicted placement, where a lower count indicates fewer routing conflicts and a cleaner signal topology.

Net Separation (NS) refers to the average spatial distance between electrically connected pins across all nets, measuring how well the placement clusters connected components together.

Predicted Routed Wirelength (PRWL) refers to the actual total wirelength after routing via FreeRouting (Freerouting, 2024), capturing the true routing cost of the predicted placement.

Routability Ratio (RR) refers to the fraction of nets successfully routed by FreeRouting (Freerouting, 2024), directly measuring whether the predicted placement supports complete electrical connectivity.

Number of Vias (#Vias) refers to the total number of vias introduced during routing, where a lower count indicates a more layer-efficient placement with fewer inter-layer transitions.

Schematic Relational Distance ($\mathcal{D}_{sch}$) refers to the average pairwise relative distance deviation between the predicted layout and the schematic diagram over all component pairs within the same subcircuit, measuring whether the predicted placement preserves the visual subcircuit topology of the circuit schematic.

Reference Distance ($\mathcal{D}_{ref}$) refers to the average pairwise relative distance deviation between the predicted layout and the expert-designed reference layout, measuring the degree to which the predicted placement aligns with professional component arrangement decisions.

Constraint Satisfaction Rate (CSR) refers to the ratio of evaluated geometric and routing constraints satisfied at the final placement iteration, measuring the overall correctness of the agentic placement outcome.

Step Efficiency (SE) refers to the average number of tool-use steps required to reach a valid placement, where a lower value indicates more efficient constraint-aware spatial reasoning.

Placement Refinement Quality (PRQ) refers to the fraction of iterations in which the placement improves over the previous step, capturing whether the agent consistently makes progress toward a valid placement across iterations.

B.2 Evaluation Metrics

Once the component placement is predicted, we compute the following evaluation metrics across four complementary dimensions.

Overlap (OO). Given a predicted placement $\mathcal{P}$ with components $\{c_1, c_2, \ldots, c_N\}$, the normalized overlapping area between components is computed as:

$$\mathrm{OO}(\mathcal{P}) = \frac{\sum_{i<j} \mathrm{Area}(c_i \cap c_j)}{\sum_i \mathrm{Area}(c_i)} \tag{4}$$

where $c_i \cap c_j$ denotes the intersecting area between components i and j. Lower values indicate fewer physical conflicts between components. We only compute for components in the same layer.

Out-of-Boundary (OoB). The ratio of component area placed outside the board boundary B is computed as:

$$\mathrm{OoB}(\mathcal{P}) = \frac{\sum_i \mathrm{Area}(c_i \setminus B)}{\sum_i \mathrm{Area}(c_i)} \tag{5}$$

where $c_i \setminus B$ denotes the area of component i outside the board boundary. Lower values indicate better spatial containment within the board.

Half-Perimeter Wirelength (HPWL). For a set of nets $\mathcal{N}$, HPWL is computed as:

$$\mathrm{HPWL}(\mathcal{P}) = \sum_{n \in \mathcal{N}} \Big(\max_{i \in n} x_i - \min_{i \in n} x_i + \max_{i \in n} y_i - \min_{i \in n} y_i \Big) \tag{6}$$

where (x_i, y_i) denotes the position of pin i in net n. Lower HPWL indicates more compact and routing-friendly component placement.

Net Crossing (NC) refers to the total number of intersections between net connections in the predicted placement, where a lower count indicates fewer routing conflicts and a cleaner signal topology:

$$\mathrm{NC}(\mathcal{P}) = \sum_{n_i \in \mathcal{N}} \sum_{n_j \in \mathcal{N}, j>i} \mathbb{1}\left[e_{n_i} \cap e_{n_j} \neq \emptyset \right] \tag{7}$$

where e_{n_i} and e_{n_j} denote the straight-line connections of nets n_i and n_j respectively, and $\mathbb{1}[\cdot]$ is the indicator function that equals 1 when two net connections intersect.

Net Separation (NS). NS measures the average spatial distance between electrically connected pins across all nets:

$$\mathrm{NS}(\mathcal{P}) = \frac{1}{|\mathcal{N}|} \sum_{n \in \mathcal{N}} \frac{1}{|n|} \sum_{i \in n} \|p_i - \bar{p}_n\|_2 \quad (8)$$

where p_i denotes the position of pin i and $\bar{p}_n$ denotes the centroid of net n.

Routability Ratio (RR). RR is defined as the fraction of successfully routed nets obtained via FreeRouting (Freerouting, 2024):

$$\mathrm{RR}(\mathcal{P}) = \frac{|\mathcal{N}_{routed}|}{|\mathcal{N}|} \quad (9)$$

where $|\mathcal{N}_{routed}|$ denotes the number of successfully routed nets and $|\mathcal{N}|$ denotes the total number of nets.

Predicted Routed Wirelength (PRWL) refers to the actual total wirelength after routing via FreeRouting (Freerouting, 2024), capturing the true routing cost of the predicted placement:

$$\mathrm{PRWL}(\mathcal{P}) = \sum_{n \in \mathcal{N}} \sum_{e \in \mathcal{R}_n} \|e\|_2 \quad (10)$$

where $\mathcal{R}_n$ denotes the set of routed wire segments for net n, and $\|e\|_2$ denotes the Euclidean length of wire segment e.

Number of Vias (#Vias). #Vias records the total number of vias introduced during routing, where a lower count indicates a more layer-efficient placement.

Schematic Relational Distance ($\mathcal{D}_{sch}$). Since PCB component placement does not admit a unique ground truth solution, we measure the visual relative distance between the predicted layout and the schematic diagram $\mathcal{S}$:

$$\mathcal{D}_{sch}(\mathcal{P}, \mathcal{S}) = \frac{1}{|\mathcal{G}|} \sum_{(i,j) \in \mathcal{G}} |d_{\mathcal{P}}(i,j) - d_{\mathcal{S}}(i,j)| \quad (11)$$

where $\mathcal{G}$ denotes the set of component pairs within the same subcircuit, and $d_{\mathcal{P}}(i,j)$, $d_{\mathcal{S}}(i,j)$ denote the pairwise distances between components i and j in the predicted layout and schematic diagram respectively.

Reference Distance ($\mathcal{D}_{ref}$). We further measure the component-level relative placement distance against the expert-designed reference layout $\mathcal{R}$:

$$\mathcal{D}_{ref}(\mathcal{P}, \mathcal{R}) = \frac{1}{|\mathcal{G}|} \sum_{(i,j) \in \mathcal{G}} |d_{\mathcal{P}}(i,j) - d_{\mathcal{R}}(i,j)| \quad (12)$$

where $d_{\mathcal{R}}(i,j)$ denotes the pairwise distance between components i and j in the reference layout. Because lower D_{ref} indicates closer agreement, we report the bounded reference-layout similarity

$$S_{\mathrm{ref}}(\mathcal{P}, \mathcal{R}) = \frac{1}{1 + D_{\mathrm{ref}}(\mathcal{P}, \mathcal{R})}. \quad (13)$$

Higher S_{ref} indicates closer agreement with the expert layout. This metric is an auxiliary spatial diagnostic and does not measure electrical correctness.

Constraint Satisfaction Rate (CSR). CSR measures the ratio of evaluated geometric and routing constraints satisfied at the final placement iteration T:

$$\mathrm{CSR}(\mathcal{P}_T) = \frac{|\mathcal{C}_{sat}|}{|\mathcal{C}|} \quad (14)$$

where $|\mathcal{C}_{sat}|$ and $|\mathcal{C}|$ denote the number of satisfied and total constraints respectively.

Placement Refinement Quality (PRQ). PRQ measures whether successive placement iterations progressively reduce constraint violations:

$$\mathrm{PRQ} = \frac{1}{T-1} \sum_{t=1}^{T-1} \mathbb{1}\left[\mathcal{V}(\mathcal{P}_{t+1}) < \mathcal{V}(\mathcal{P}_t)\right] \quad (15)$$

where $\mathcal{V}(\mathcal{P}_t)$ denotes the total number of constraint violations at iteration t.

Functional-Grouping AUC. For each board, component pairs are labeled positive when they belong to the same schematic functional group and negative otherwise. We rank pairs using negative Euclidean placement distance and compute ROC AUC. Higher AUC indicates that same-group components tend to be placed closer together. This metric allows multiple coordinate solutions but remains a relational proxy rather than electrical validation.

Step Efficiency (SE). SE measures the average number of tool-use steps required to reach a valid placement, where a lower value indicates more efficient constraint-aware reasoning.

C Experiment

C.1 Additional Analysis of Agentic Tool Use

The agentic study evaluates three complementary feedback channels. Visualization provides a rendered image of the current placement; Geo. Check provides explicit information about component-overlap and boundary violations; and Routing Score provides HPWL-based routing feedback. The All Tools condition makes all three feedback channels available simultaneously.

The results in Table 3 demonstrate that feedback utility depends on both the model and the optimization objective. For the three models examined here, the All Tools condition achieves the lowest overlap error. Claude-Sonnet-4.6 obtains an overlap error of 0.004 with All Tools, compared with 0.029 using Visualization, 0.011 using Geo. Check, and 0.055 using Routing Score. Ministral-14B similarly achieves its lowest overlap error with All Tools (0.065), compared with 0.130, 0.177, and 0.124 under Visualization, Geo. Check, and Routing Score, respectively. GPT-5-mini follows the same pattern, achieving an overlap error of 0.024 with All Tools, compared with 0.028 using Visualization, 0.049 using Geo. Check, and 0.095 using Routing Score. These results suggest that jointly providing visual, geometric, and routing feedback can facilitate overlap correction by allowing models to identify and repair layout conflicts using complementary signals.

However, the All Tools condition does not consistently achieve the best performance across other evaluation metrics. For example, Visualization produces the lowest out-of-boundary ratio for Claude-Sonnet-4.6 and Ministral-14B, whereas Routing Score produces the lowest HPWL and highest routability ratio for Ministral-14B. Similarly, although All Tools gives GPT-5-mini its lowest overlap error and HPWL, Visualization yields a substantially lower out-of-boundary ratio. These results reveal trade-offs among geometric legality, routability, and relational placement quality: optimizing one criterion may degrade another.

Overall, combining all feedback channels is effective for reducing overlap for the models analyzed here, but it is not uniformly optimal across models and evaluation objectives. Simultaneous feedback may introduce competing objectives or cause a model to prioritize one type of violation over another. These findings motivate adaptive agentic systems that select feedback channels according to the current layout state, target metric, and model capability. Because the successfully parsed sample subsets may differ across conditions, we interpret these results as within-agentic diagnostic comparisons rather than evidence of uniform improvement over one-shot placement.

C.2 Prompt Template

We use task-specific prompt templates to guide the model under different PCB layout settings. Rather than using a single generic instruction, each task is associated with a tailored prompt that consolidates the placement objective, board boundary constraints, component block dimensions, netlist connectivity, allowed rotation codes, placement rules, and required output format into a single unified instruction. The model is expected to parse all spatial and structural information within this self-contained prompt and produce a valid, constraint-satisfying placement in a single pass. In the following, we present representative prompt examples used in our layout evaluation framework.

Oneshot Evaluation Prompt Template. We design five prompt variants to systematically evaluate the impact of different auxiliary inputs on LLM-based PCB placement. • **Multimodal (Schematic Image-Guided)** This variant extends the baseline by supplying rendered schematic images as additional visual input, allowing the model to leverage visual circuit structure as a reference for functional grouping and relative placement hints. • **Graph-Augmented (Schematic Graph-Guided)** This variant injects a secondary schematic graph JSON encoding symbol-level connectivity with spatial distance weights on pin-pin edges, guiding the model to keep strongly-related components closer together on the board. • **Full-Graph (Spatial-Semantic Graph-Guided)** This variant further enriches the graph representation by retaining per-node semantic attributes for symbols, pins, nets, and text annotations alongside the spatial edge weights, enabling the model to jointly exploit both structural connectivity and semantic context. • **Few-Shot (Example-Guided)** This variant augments the baseline with one concrete placement example demonstrating an effective layout strategy, instructing the model to mimic the observed spatial reasoning patterns while adapting coordinates to the current board and component set.

#Oneshot_prompt (Baseline Task):
You are an expert PCB layout engineer. Your task is to place all components
on a PCB board optimally.

Board Boundary:
Overall Board boundary extents: (x_0, y_0) to (x_1, y_1), size W mm $\times$ H mm.
Board outline is defined only by board_boundary.dxf_details.
Use this outline to constrain all placements inside the board.
Board boundary dxf_details (JSON): {dxf_details_json}

Component Block Dimensions:
For each component, block_dimension is a list of blocks inside the component that cannot overlap with blocks from other components. Each block has: position (relative to component center), width, length, type (smd/pad/hole), and layer (front/back/through).
Allowed rotation codes: R0: front side, 0° R180: front side, 180° R270: front side, 270°
MR0: back side (mirrored), 0° MR180: back side (mirrored), 180° MR270: back side (mirrored), 270°

Components (name and block dimensions):
- COMP_NAME: type (layer) W mm $\times$ L mm at (x, y)
- ...

Netlist (connections):
- Net 'NET_NAME': element:pad, element:pad, ...
- ...

Placement Rules:
1. Coordinates must be inside the board boundary
$(x_0 \leq x \leq x_1, \ y_0 \leq y \leq y_1)$.
2. **CRITICAL**: Block dimensions from different components must NOT overlap. Blocks within the same component are already non-overlapping by design.
3. Components CAN share the same center position, but their
block_dimensions must not overlap in 2D.
4. Choose placement side (R* vs MR*) to avoid block collisions while respecting layer constraints.

Output Format (strictly follow this):
Return ONLY a valid JSON object, no explanation, no markdown fences.
{
"placements": [
{"name": "COMP", "x": X, "y": Y, "rotation": "R0|···"},
...
]
}
Every component must appear exactly once.
Coordinates are in millimeters.
Place component CENTER at (x, y).

#Multimodal_prompt (Schematic Image-Guided Layout Task):
You are an expert PCB layout engineer. Your task is to place all components
on a PCB board optimally.

You are also given schematic image(s) as multimodal input in this request.
Treat them as reference for functional grouping and relative placement hints.

Board Boundary:
Overall Board boundary extents: (x_0, y_0) to (x_1, y_1), size W mm $\times$ H mm. Board outline is defined only by board_boundary.dxf_details. Use this outline to constrain all placements inside the board. Board boundary dxf_details (JSON): {dxf_details_json}

Component Block Dimensions:
For each component, block_dimension is a list of blocks inside the component that cannot overlap with blocks from other components. Each block has: position (relative to component center), width, length, type (smd/pad/hole), and layer (front/back/through).
Allowed rotation codes: R0: front side, 0° R180: front side, 180° R270: front side, 270°
MR0: back side (mirrored), 0° MR180: back side (mirrored), 180° MR270: back side (mirrored), 270°
Components (name and block dimensions):
- COMP_NAME: type (layer) W mm $\times$ L mm at (x, y)
- ...

Netlist (connections):
- Net 'NET_NAME': element:pad, element:pad, ...
- ...

Placement Rules:
1. Coordinates must be inside the board boundary
$(x_0 \leq x \leq x_1, \ y_0 \leq y \leq y_1)$.
2. **CRITICAL**: Block dimensions from different components must NOT overlap. Blocks within the same component are already non-overlapping by design.
3. Components CAN share the same center position, but their block_dimensions must not overlap in 2D.
4. Choose placement side (R* vs MR*) to avoid block collisions
while respecting layer constraints.
5. Use the provided schematic image(s) as reference for functional grouping and relative placement hints between connected components.

Output Format (strictly follow this):
Return ONLY a valid JSON object, no explanation, no markdown fences.
{
"placements": [

{“name”: “COMP”, “x”: X, “y”: Y, “rotation”: “R0|···”},
···
]
}
Every component must appear exactly once.
Coordinates are in millimeters.
Place component CENTER at (x, y).

#Graph-Augmented_prompt (Schematic Graph-Guided Layout Task):
You are an expert PCB layout engineer. Your task is to place all components on a PCB board optimally.

You are also provided with a schematic graph (JSON) extracted from the schematic file, which encodes symbol-level connectivity and relative spatial relationships among components. Use it as an auxiliary signal to guide functional grouping and placement decisions.

Board Boundary:
Overall Board boundary extents: (x_0, y_0) to (x_1, y_1), size W mm $\times$ H mm. Board outline is defined only by board_boundary.dxf_details. Use this outline to constrain all placements inside the board. Board boundary dxf_details (JSON): {dxf_details_json}

Component Block Dimensions:
For each component, block_dimension is a list of blocks inside the component that cannot overlap with blocks from other components. Each block has: position (relative to component center), width, length, type (smd/pad/hole), and layer (front/back/through).
Allowed rotation codes:
R0: front side, 0° R180: front side, 180°
R270: front side, 270°
MR0: back side (mirrored), 0° MR180: back side (mirrored), 180° MR270: back side (mirrored), 270°

Components (name and block dimensions):
- COMP_NAME: type (layer) W mm $\times$ L mm at (x, y)
- ···

Netlist (connections):
- Net ‘NET_NAME’: element:pad, element:pad, ...
- ···

Secondary Schematic Graph Input (JSON):
{sch_graph_json}

How to interpret this schematic graph:
- Netlist information extracted from the schematic (.sch) file.
- Edges with type symbol_pin indicate which symbol each pin belongs to.
- Edges with type pin-pin indicate connectivity among symbols through their pins.
- For pin-pin edges, the weight is a relative distance signal on the schematic image. Smaller weight means symbols are relatively closer on the schematic.
- Use this relative-distance signal to improve placement decisions: keep strongly-related symbols relatively close when possible.

Placement Rules:
1. Coordinates must be inside the board boundary $(x_0 \le x \le x_1,\ y_0 \le y \le y_1)$.
2. **CRITICAL**: Block dimensions from different components must NOT overlap. Blocks within the same component are already non-overlapping by design.
3. Components CAN share the same center position, but their block_dimensions must not overlap in 2D.
4. Choose placement side (R* vs MR*) to avoid block collisions while respecting layer constraints.

Output Format (strictly follow this):
Return ONLY a valid JSON object, no explanation, no markdown fences.
{
“placements”: [
{“name”: “COMP”, “x”: X, “y”: Y, “rotation”: “R0|···”},
···
]
}
Every component must appear exactly once.
Coordinates are in millimeters.
Place component CENTER at (x, y).

#Full-Graph_prompt (Spatial-Semantic Graph-Guided Layout Task):
You are an expert PCB layout engineer. Your task is to place all components on a PCB board optimally.
You are also provided with a secondary schematic graph (JSON) extracted from the schematic file, which jointly encodes symbol-level connectivity with relative spatial distance weights and per-node semantic attributes for symbols, pins, nets, and text annotations. Use both signals together as auxiliary guidance for functional grouping and placement decisions.

Board Boundary:
Overall Board boundary extents: (x_0, y_0) to (x_1, y_1), size W mm $\times$ H mm. Board outline is defined only by board_boundary.dxf_details. Use this outline to constrain all placements inside the board. Board boundary dxf_details (JSON): {dxf_details_json}

Component Block Dimensions:
For each component, block_dimension is a list of blocks inside the component that cannot overlap with blocks from other components. Each block has: position (relative to component center), width, length, type (smd/pad/hole), and layer (front/back/through).
Allowed rotation codes:
R0: front side, 0° R180: front side, 180° R270: front side, 270°
MR0: back side (mirrored), 0° MR180: back side (mirrored), 180° MR270: back side (mirrored), 270°

Components (name and block dimensions):
- COMP_NAME: type (layer) W mm $\times$ L mm at $(x,\ y)$
- ...

Netlist (connections):
- Net 'NET_NAME': element:pad, element:pad, ...
- ...

Secondary Schematic Graph Input (JSON):
{sch_graph_json}

How to interpret this schematic graph:
- Extracted from the schematic (.sch) file; contains both structural
edges and per-node semantic attributes.
- Edges with type symbol_pin indicate which symbol each pin belongs to.
- Edges with type pin-pin indicate connectivity among symbols
through their pins.
- For pin-pin edges, the weight is a relative distance signal on the schematic image. Smaller weight means symbols are relatively closer on the schematic.
- Use this relative-distance signal to improve placement decisions:
keep strongly-related symbols relatively close when possible.
- Node attributes (for symbols/pins/nets) and text annotations provide
semantic context; use them as auxiliary hints for functional grouping.

Placement Rules:
1. Coordinates must be inside the board boundary
$(x_0 \le x \le x_1,\ y_0 \le y \le y_1)$.
2. **CRITICAL**: Block dimensions from different components must NOT overlap.
Blocks within the same component are already non-overlapping by design.
3. Components CAN share the same center position, but their
block_dimensions must not overlap in 2D.
4. Choose placement side (R* vs MR*) to avoid block collisions
while respecting layer constraints.

Output Format (strictly follow this):
Return ONLY a valid JSON object, no explanation, no markdown fences.

```
{
"placements": [
{"name": "COMP", "x": X, "y": Y, "rotation": "R0|···"},
...
]
}
```

Every component must appear exactly once.
Coordinates are in millimeters.
Place component CENTER at (x, y).

#Few-Shot_prompt (Example-Guided Layout Task):
You are an expert PCB layout engineer. Your task is to place all components on a PCB board optimally.

You are also provided with three placement examples that demonstrate effective layout strategies. Mimic the placement reasoning and spatial patterns shown in the examples, while adapting exact coordinates to the current board and component set.

Board Boundary:
Overall Board boundary extents: $(x_0,\ y_0)$ to $(x_1,\ y_1)$, size W mm $\times$ H mm.
Board outline is defined only by board_boundary.dxf_details.
Use this outline to constrain all placements inside the board.
Board boundary dxf_details (JSON):
{dxf_details_json}

Component Block Dimensions:
For each component, block_dimension is a list of blocks inside the
component that cannot overlap with blocks from other components.
Each block has: position (relative to component center), width, length,
type (smd/pad/hole), and layer (front/back/through).
Allowed rotation codes:
R0: front side, 0° R180: front side, 180° R270: front side, 270°
MR0: back side (mirrored), 0° MR180: back side (mirrored), 180° MR270: back side (mirrored), 270°

Components (name and block dimensions):
- COMP_NAME: type (layer) W mm $\times$ L mm at $(x,\ y)$
- ...

Netlist (connections):
- Net 'NET_NAME': element:pad, element:pad, ...
- ...

Placement Rules:
1. Coordinates must be inside the board boundary
$(x_0 \le x \le x_1,\ y_0 \le y \le y_1)$.
2. **CRITICAL**: Block dimensions from different components must NOT overlap.

```
Blocks within the same component are already
non-overlapping by design.
3. Components CAN share the same center
position, but their
block_dimensions must not overlap in 2D.
4. Choose placement side (R* vs MR*) to
avoid block collisions
while respecting layer constraints.

Few-Shot Examples (mimic strategy, adapt
coordinates to current board):
Example 1: Two ICs with nearby decoupling
capacitors
Input: Board 40×30 mm. Components: U1(MCU),
U2(SENSOR), C1, C2, J1.
Nets:        U1:VCC-C1:1,      U2:VCC-C2:1,
U1:SCL-U2:SCL, U1:IO-J1:1.
Output: {"placements": [{U1 at (16,16,R0)},
{U2 at (24,16,R0)},
{C1 at (16,12.5,R0)}, {C2 at (24,12.5,R0)},
{J1 at (35,15,R180)}]}
...

Output Format (strictly follow this):
Return ONLY a valid JSON object, no
explanation, no markdown fences.
{
"placements": [
{"name": "COMP", "x": X, "y": Y, "rotation":
"R0|..."},
...
]
}
Every component must appear exactly once.
Coordinates are in millimeters.
Place component CENTER at (x, y).
```

Agentic Evaluation Prompt Template. We design four prompt variants to systematically evaluate the impact of different feedback signals on LLM-based agentic PCB placement. • **HPWL-Guided (Routability Risk Feedback)** This variant replaces the raw half-perimeter wirelength metric with a learned routability risk score as the iterative feedback signal, allowing the agent to optimize toward congestion-aware placement objectives rather than purely geometric wire minimization. • **Overlap-Guided (Constraint Violation Feedback)** This variant exposes only the overlap inspection tool during each iteration, providing the agent with binary constraint violation signals that directly indicate whether placed components physically intersect, guiding the model to prioritize feasibility over optimality in its revision decisions. • **Boundary-Guided (Out-of-Boundary Feedback)** This variant restricts feedback to out-of-boundary detection results, supplying the agent with spatial legality signals that indicate whether components exceed board extents, encouraging the model to learn boundary-respecting placement revisions through iterative self-correction. • **Multimodal-Guided (Visual Feedback)** This variant furnishes the agent with rendered placement images as its sole inspection signal at each iteration, requiring the model to interpret spatial quality and identify layout deficiencies directly from visual observations rather than from structured numeric or symbolic outputs. • **Full-Feedback (All-Tools Adaptive)** This variant grants the agent unrestricted access to all available inspection tools — routability risk, overlap detection, out-of-boundary checking, and visual rendering — at each iteration, allowing the model to autonomously select the most informative feedback signals based on its current placement state and self-assessed quality, rather than being constrained to a single fixed feedback modality throughout the optimization loop.

Diagnostic analysis protocol. Board complexity is measured by component count, while placement density is defined as the total area of pads, SMDs, and holes divided by the valid board area. For the component-count analysis, boards are grouped into six fixed intervals: 1–20, 21–40, 41–60, 61–100, 101–160, and 161–327 components. For the placement-density analysis, boards are grouped into 0–10%, 10–20%, 20–30%, 30–40%, 40–60%, and $\geq$60% intervals. For each model and interval, we report the mean normalized overlap area over successfully parsed placements. Empty, invalid, or unparseable model responses are excluded; intervals containing no valid placements are reported as unavailable. For each model, the relative increase from the lowest to the highest interval is computed a $100(\mathrm{OO}_{Q4} - \mathrm{OO}_{Q1})/\mathrm{OO}_{Q1}$. We report the median of these model-specific relative increases across models.

```
#Agentic_prompt Variation 1 – HPWL-Guided
(Routability Risk Feedback):
You are an expert PCB layout engineer and
placement-decision agent.
You must do two jobs in one response:
1.    Produce the next candidate PCB
placement.
2.    Decide which repair actions and
inspection tools should be used after this
candidate is saved and rendered.

Strict priority order:
1. Return schema-valid JSON only. Include
every required component exactly once.
2. Repair deterministic geometry problems
first: out_of_boundary, then overlap.
3.   Use HPWL/routability risk to guide
```

compactness only after the placement is geometrically valid.

Board Boundary:
Extents: $x \in [x_0, x_1]$, $y \in [y_0, y_1]$ millimeters. Exact outline primitives are in board_boundary.dxf_details (summary: {dxf_summary}).

Iteration:
Current iteration: {iteration} / Maximum iterations: {max_iterations}.
If iteration == max_iterations, return your best candidate and do not request unnecessary actions.

Required components (exactly once each):
{names_json}

Component Block Dimensions:
The placement coordinate (x, y) is the component CENTER in millimeters.
block_dimension lists the physical blocks/pads/holes inside each component.
Blocks from different components must not overlap on the same board side.
- COMP_NAME: pad:smd (front) W mm $\times$ L mm at (bx, by) rot=R0; ...

Source Rotation Operations (read-only source orientation records):
{source_rotation_operations_json}

Netlist:
- Net 'NET_NAME': element:pad, ...

Safe Center Range Hints (keep center inside these ranges per rotation):
{fit_hints_json}

Allowed rotation codes:
R0: front, 0° R90: front, 90° R180: front, 180° R270: front, 270°
MR0: back mirrored, 0° MR90: back mirrored, 90° MR180: back mirrored, 180° MR270: back mirrored, 270°

Previous Context:
{previous_context_json}

Recent History:
{history_summary_json}

How to use previous context:
- previous_inspection_results are observations from the previous saved placement.
- If previous hpwl is high and geometry is clean, apply improve_routability to reduce net spread.
- **HPWL slot is replaced by routability_risk**: a lower score is better; compare current vs. previous score before requesting another routability action.
- If geometry is still invalid, ignore routability risk until out_of_boundary and overlap pass.

Available actions:
- "fix_boundary": repair blocks outside the board outline.
- "fix_overlap": repair block overlaps between different components.
- "improve_routability": reduce routability risk / net spread only after geometry is clean.
- "adjust_visual_layout": improve visual plausibility (not applicable in this variation; omit or leave empty).
- Return action=[] if no further iteration is needed.

Available inspection_tools (exactly one allowed):
- "hpwl" — reports routability_risk score for the current candidate. Lower is better.

Placement rules: (same as baseline — boundary, overlap, netlist compactness)

Decision rules:
1. If previous hpwl (routability_risk) increased, action must include "improve_routability" with a concrete move plan.
2. quality_score must not exceed 0.95 while known geometry violations remain.
3. visual_feedback: say "No previous visual image was provided." (visualize is not available in this variation).

Output Format (strictly follow this):
Return ONLY a valid JSON object, no markdown fences.
{
"placements": [{. . . }],
"decision": {
"action": [ ],
"inspection_tools": ["hpwl"],
"quality_score": 1.0,
"next_action_description": ". . . ",
"feedback": ". . . ",
"thought": ". . . ",
"visual_feedback": "No previous visual image was provided."
}
}

#Agentic_prompt Variation 2 — Overlap-Guided (Constraint Violation Feedback):
You are an expert PCB layout engineer and placement-decision agent.
You must do two jobs in one response:
1. Produce the next candidate PCB placement.
2. Decide whether the current candidate needs a repair iteration based exclusively on block overlap violations.

Strict priority order:
1. Return schema-valid JSON only. Include every required component exactly once.
2. Eliminate all block overlaps between different components.

3. Boundary compliance and routability are secondary; focus on overlap-free placement above all else.

Board Boundary:
Extents: $x \in [x_0, x_1]$, $y \in [y_0, y_1]$ millimeters.
Outline summary: {dxf_summary}.

Iteration:
Current iteration: {iteration} / Maximum iterations: {max_iterations}.

Required components, Component Block Dimensions, Source Rotation Operations, Netlist, Safe Center Range Hints, Allowed rotation codes:
(identical to baseline — see shared section above)

Previous Context:
{previous_context_json}

Recent History:
{history_summary_json}

How to use previous context:
- If previous overlap tool returned ok: false, the errors list names the exact component pairs that collide; move those pairs apart and include "fix_overlap" in decision.action.
- Do not repeat a placement that failed overlap inspection unless you have explicitly changed the relevant component positions.

Available actions:
- "fix_overlap": repair block overlaps between different components.
- "fix_boundary": secondary; use only if overlap is already clean.
- "improve_routability": secondary; use only if overlap is clean.
- Return action=[] if overlap is expected to pass.

Available inspection_tools (exactly one allowed):
- "overlap" — deterministic local block overlap detection. Returns per-pair error messages when ok: false.

Decision rules:
1. quality_score must not exceed 0.95 while any overlap error remains unresolved.
2. visual_feedback: say "No previous visual image was provided." (visualize is not available in this variation).
3. feedback must quote the specific component names from the previous overlap error list and describe the move applied.

Output Format (strictly follow this):
Return ONLY a valid JSON object, no markdown fences.
{
"placements": [{. . . }],
"decision": {
"action": [ ],
"inspection_tools": ["overlap"],
"quality_score": 1.0,
"next_action_description": ". . . ",
"feedback": ". . . ",
"thought": ". . . ",
"visual_feedback": "No previous visual image was provided."
}
}

#Agentic_prompt Variation 3 — Boundary-Guided (Out-of-Boundary Feedback):
You are an expert PCB layout engineer and placement-decision agent.
You must do two jobs in one response:
1. Produce the next candidate PCB placement.
2. Decide whether the current candidate needs a repair iteration based exclusively on out-of-boundary violations.

Strict priority order:
1. Return schema-valid JSON only. Include every required component exactly once.
2. Keep every transformed block strictly inside the board outline polygon.
3. Overlap avoidance and routability are secondary; spatial legality is the single convergence criterion in this variation.

Board Boundary:
Extents: $x \in [x_0, x_1]$, $y \in [y_0, y_1]$ millimeters.
Exact polygon outline is in board_boundary.dxf_details (summary: {dxf_summary}).
CRITICAL: use the polygon outline, not just the bounding box, for boundary legality; a component center inside the bbox can still have blocks outside an irregular outline.

Iteration:
Current iteration: {iteration} / Maximum iterations: {max_iterations}.

Required components, Component Block Dimensions, Source Rotation Operations, Netlist, Safe Center Range Hints, Allowed rotation codes:
(identical to baseline — see shared section above)

Previous Context:
{previous_context_json}

Recent History:
{history_summary_json}

How to use previous context:
- If previous out_of_boundary tool returned ok: false, the errors list reports each violating block with its oob_area value; move the component center into the safe_center_ranges for its rotation and include "fix_boundary" in decision.action.
- SAFE_CENTER_RANGE_HINTS gives pre-computed

safe center ranges per rotation per component; prefer those ranges to avoid polygon violations.

Available actions:
- "fix_boundary": move violating component centers into their safe range so all blocks lie inside the board polygon.
- "fix_overlap": secondary; use only if boundary is already clean.
- "improve_routability": secondary; use only if boundary is clean.
- Return action=[] if out_of_boundary is expected to pass.

Available inspection_tools (exactly one allowed):
- "out_of_boundary" — deterministic polygon boundary detection. Reports per-block oob_area when ok: false.

Decision rules:
1. quality_score must not exceed 0.95 while any boundary error remains unresolved.
2. visual_feedback: say "No previous visual image was provided." (visualize is not available in this variation).
3. feedback must state the violating component names and which rotation + center coordinate was chosen to move them inside bounds.

Output Format (strictly follow this):
Return ONLY a valid JSON object, no markdown fences.
{
"placements": [{. . . }],
"decision": {
"action": [ ],
"inspection_tools": ["out_of_boundary"],
"quality_score": 1.0,
"next_action_description": ". . . ",
"feedback": ". . . ",
"thought": ". . . ",
"visual_feedback": "No previous visual image was provided."
}
}

#Agentic_prompt Variation 4 — Multimodal-Guided (Visual Feedback):
You are an expert PCB layout engineer and placement-decision agent.
You must do two jobs in one response:
1. Produce the next candidate PCB placement.
2. Decide whether the current candidate needs a visual improvement iteration based exclusively on the rendered placement image.

Strict priority order:
1. Return schema-valid JSON only. Include every required component exactly once.
2. Use the attached rendered image from the previous iteration as the primary feedback signal; analyze spatial density, component clustering, and obvious visual irregularities.
3. Numeric metrics (overlap counts, boundary violations, HPWL) are not available in this variation; all quality judgements must be derived from visual observation.

Board Boundary:
Extents: $x \in [x_0, x_1]$, $y \in [y_0, y_1]$ millimeters.
Outline summary: {dxf_summary}.

Iteration:
Current iteration: {iteration} / Maximum iterations: {max_iterations}.
If a rendered image is attached, it belongs to iteration {iteration−1}; your visual analysis applies to that image and your new placement must address any deficiencies you observe.

Required components, Component Block Dimensions, Source Rotation Operations, Netlist, Safe Center Range Hints, Allowed rotation codes:
(identical to baseline — see shared section above)

Previous Context (including attached rendered image):
{previous_context_json}
Note: previous_visualize_note — a rendered image from the previous iteration is attached. Any visual problems you identify belong to that previous image; write your analysis in decision.visual_feedback for the current iteration and update the new placement accordingly.

Recent History:
{history_summary_json}

How to use previous context:
- Inspect the attached rendered image for crowding, isolated components, long diagonal ratsnest lines, and unbalanced empty zones.
- Move components to address the visual deficiencies you identify; include "adjust_visual_layout" in decision.action.
- Do not request overlap or out_of_boundary tools; they are not available in this variation.

Available actions:
- "adjust_visual_layout": improve visual plausibility using the previous rendered image.
- Return action=[] if the visual layout appears satisfactory.

Available inspection_tools (exactly one allowed):
- "visualize" — render the current candidate as a PNG image. The rendered image becomes the visual feedback for the next iteration.

Decision rules:
1. visual_feedback is mandatory and must

describe specific layout deficiencies observed in the previous image (e.g., component clustering, large empty zones, suspected overlapping bodies).
2. If no image was provided (first iteration), write “No previous visual image was provided.” and propose an initial spread layout.
3. quality_score reflects your visual confidence only; do not assume geometry correctness.

Output Format (strictly follow this):
Return ONLY a valid JSON object, no markdown fences.
{
“placements”: [{. . . }],
“decision”: {
“action”: [ ],
“inspection_tools”: [“visualize”],
“quality_score”: 0.9,
“next_action_description”: “. . . ”,
“feedback”: “. . . ”,
“thought”: “. . . ”,
“visual_feedback”: “Observed crowding near board center . . . ”
}
}

#Agentic_prompt Variation 5 — Full-Feedback (All-Tools Adaptive):
You are an expert PCB layout engineer and placement-decision agent.
You must do two jobs in one response:
1. Produce the next candidate PCB placement.
2. Autonomously select the most informative inspection tools and repair actions based on your current assessment of the placement quality.

Strict priority order:
1. Return schema-valid JSON only. Include every required component exactly once.
2. Repair deterministic geometry problems first: out_of_boundary, then overlap.
3. Use visual feedback to improve obvious layout issues after deterministic geometry is addressed.
4. Use HPWL/netlist compactness only after the placement is geometrically valid.

Board Boundary:
Extents: $x \in [x_0, x_1]$, $y \in [y_0, y_1]$ millimeters. Exact outline primitives are in board_boundary.dxf_details (summary: {dxf_summary}).

Iteration:
Current iteration: {iteration} / Maximum iterations: {max_iterations}.
If iteration == max_iterations, return your best candidate and do not request unnecessary actions.

Required components, Component Block Dimensions, Source Rotation Operations, Netlist, Safe Center Range Hints, Allowed rotation codes:
(identical to baseline — see shared section above)

Previous Context:
{previous_context_json}
Note: if ”visualize” was selected in the previous iteration, a rendered image is attached. Any visual problems you identify belong to that previous image; write your analysis in decision.visual_feedback and update the new placement accordingly.

Recent History:
{history_summary_json}

How to use previous context:
- If previous overlap failed, move the involved components apart and include ”fix_overlap” in decision.action.
- If previous out_of_boundary failed, move the violating component center into its safe range and include ”fix_boundary”.
- If previous hpwl is high but geometry is still invalid, ignore hpwl until geometry is clean.
- If a rendered image is attached, inspect it now and summarize visual deficiencies in decision.visual_feedback.

Available actions:
- ”fix_boundary”: repair blocks outside the board outline.
- ”fix_overlap”: repair block overlaps between different components.
- ”adjust_visual_layout”: improve visual plausibility using the previous rendered image.
- ”improve_routability”: reduce HPWL / net spread only after geometry is clean.
- Return action=[] if no further iteration is needed.

Available inspection_tools (select all that apply):
- ”visualize”: render the current candidate image for visual feedback in the next iteration.
- ”hpwl”: compute routability / compactness metric. Lower is better; compare with previous iterations.
- ”overlap”: deterministic block overlap detection.
- ”out_of_boundary”: deterministic polygon boundary detection.

Tool selection rules:
1. Select the tools needed to verify the risk areas of your current candidate.
2. If geometry is uncertain, always include overlap and out_of_boundary.
3. Add visualize when you want rendered feedback for the next iteration.
4. Add hpwl only after geometry is confirmed clean.
5. If no further iteration is needed,

```
select the minimal set that would confirm
the candidate's final quality.

Decision rules:
1.  If previous deterministic inspection
failed, action must include the matching
repair action.
2. quality_score must not exceed 0.95 while
known overlap or boundary failures remain
unresolved.
3.    visual_feedback must analyze the
previous rendered image if one was provided;
otherwise say "No previous visual image was
provided."

Output Format (strictly follow this):
Return ONLY a valid JSON object, no markdown
fences.
{
"placements": [{. . . }],
"decision": {
"action": [ ],
"inspection_tools":             ["overlap",
"out_of_boundary"],
"quality_score": 1.0,
"next_action_description":   "Finish  with
current placement if selected inspections
pass.",
"feedback":  "Concrete repair instructions
if another iteration is needed.  Empty if
done.",
"thought":  "Concise reason for selected
actions and tools.",
"visual_feedback": "Visual analysis of the
previous rendered image, or 'No previous
visual image was provided.'"
}
}
```

C.2.1 Implementation of Routing Tool

We evaluate the generated PCB placements by testing whether they can support a complete downstream routing workflow after automated post-processing. For each generated placement, we first instantiate the component locations in the Eagle board file and then export the design to the routing pipeline. The resulting board is converted into a DSN-compatible format and routed using an autorouter without manual adjustment. All routing experiments are conducted under the same design-rule settings to ensure a fair comparison across placements. Specifically, the routing setup uses a two-layer board with routing enabled on both the top and bottom layers. The via/pad rule is configured with a round shape and a radius of 16 mil. The spacing rule follows the same clearance matrix for all experiments: the default clearance is 8 mil, the boundary clearance is 10 mil, and the clearances among vias, SMD pads, pins, and copper areas are set between 8 and 10 mil according to the design-rule matrix.

We use routing success as the main functional metric, since a placement is only practically useful if the nets can be completed under the board constraints. Specifically, we report the number of successfully processed placements, the number of placements that complete routing, the routed net count, the total net count, and the routing completion ratio. We also track failures at each stage, including invalid component placement and routing failure. These metrics quantify not only whether the model can generate syntactically valid placements, but also whether those placements are geometrically and electrically feasible for downstream PCB routing.